

Supplementary Material for Stabilizing Causal Structure Learning under Heteroscedasticity

Eunjung Choi*
Dept. of Artificial Intelligence
Hanyang University
Seoul, South Korea
sinegi@hanyang.ac.kr

Seonggyeom Kim*
Dept. of Artificial Intelligence
Hanyang University
Seoul, South Korea
canino@hanyang.ac.kr

Dong-Kyu Chae
Dept. of Computer Science
Hanyang University
Seoul, South Korea
dongkyu@hanyang.ac.kr

ACM Reference Format:

Eunjung Choi, Seonggyeom Kim, and Dong-Kyu Chae. 2026. Supplementary Material for Stabilizing Causal Structure Learning under Heteroscedasticity. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*, August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 27 pages. <https://doi.org/10.1145/3770855.3818156>

A Related Work

A.1 Classical Causal Discovery Methods

The gold standard for identifying causal factors is randomized controlled trials, but in practice these face constraints of time, cost, and ethics. A more realistic setting is to learn DAGs from observational data, and research in this area has proceeded along two strands: constraint-based methods and score-based methods [16, 40]. Constraint-based methods (e.g., PC [38], FCI [7, 39]) build the causal skeleton via independence tests and determine edge orientations. Score-based methods (e.g., GES [6, 42]) maximize a score that balances data fit and model complexity and search the DAG space for an optimal graph. However these methods can identify only the Markov equivalence class [17]. Subsequently, Functional Causal Models (FCMs), which assume data generation via structural equations with direct parents and noise, were introduced to identify the true causal structure within the equivalence class, including linear SEMs [36], Additive Noise Models [33], and Post-Nonlinear models [53].

A.2 Continuous Optimization-based Methods.

However, classical methods rely on local heuristics to enforce acyclicity, which makes DAG discovery combinatorially challenging as the number of nodes increases. NOTEARS [54] reformulated the original discrete DAG constraint into a continuous optimization problem solvable by gradient descent, becoming the standard optimization approach used in subsequent DAG learning methods. Follow-up work on linear SEMs, including GOLEM [29] which studies sparsity and acyclicity and directly minimizes likelihood, and DAGMA [2] which introduces a log-det acyclicity constraint, extended this line. Subsequent methods extended these approaches

to nonlinear structures using neural function approximators, such as NOTEARS-MLP [55] and GraN-DAG [24]. Other nonlinear methods include DAG-GNN [49] that deals with various data types using variational autoencoder parameterized by graph neural network and DAG-GAN [13] that considers DAG structure learning from the perspective of distributional optimization. DAG-NoCurl [50] proposes learning the DAG on the equivalent space of weighted gradients of graph potential functions.

A.3 Different Noise Assumptions

Representative methods such as ANMs with Gaussian noise assume homoscedastic noises, meaning equal noise variance across variables and observations. Ng et al. [29] relax this by considering GOLEM-NV, where noise variances can differ by variable, while remaining constant across observations for a given variable; [24] and [32] make the same relaxation. When noise variance also differs across observations, one common approach in heteroscedasticity modeling is to relax independence between parent variables and additive noise. In the bivariate case, RECI [3] assumes a joint distribution between cause and noise; HECI [47] explicitly assumes heteroscedastic noise with variance dependent on the cause, models it as a piece-wise function of the parent, and allows only a limited number of variance values. CAFEL [23] and LOCI [22] introduce deterministic functions of the parent to adjust noise variance. To generalize to multivariate data, prior works first learn an undirected skeleton (e.g., PC [38]) then orient edges; HOST [10] estimates a causal order and then orient pair-wise directions. GraN-DAG++ [24] allows noise variances to depend on parents in realistic settings. ICDH [48] is the first to incorporate heteroscedasticity into a continuous optimization framework for DAG learning and to model the noise distribution with neural networks. More recently, SkewScore [27] proposes an ordering-based approach that leverages the skewness of conditional distributions to identify causal directions under HNMs. Similar to HOST, it first determines a causal ordering and then reconstructs the DAG via subsequent pruning. While these ordering-based methods provide effective alternatives, they are methodologically distinct from continuous optimization-based approaches such as ICDH and our method, which directly learn the DAG via gradient-based optimization with an acyclicity constraint.

*Both authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

KDD '26, Jeju Island, Republic of Korea

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2259-2/2026/08

<https://doi.org/10.1145/3770855.3818156>

A.3.1 Detailed Comparison with ICDH. Similar to Skafte et al. [37], ICDH trains the mean network and the variance network in a two-phase alternating fashion. Specifically, it first trains the mean network and W until convergence, then freezes W and updates only the variance network, repeating this cycle. As a result, during

the variance updates, the graph W is kept fixed at the value learned in the previous phase, so the parameter updates of the variance function do not directly feed back into the choice of parent variables.

In contrast, our method uses the surrogate loss defined in the main paper (Eq. (7)) and the scheduling coefficient $\lambda_{\text{reg}}(t)$ defined in the main paper (Eq. (8)) to jointly optimize the mean, the variance, and W throughout training. Since W is shared by the mean and variance networks, the choice of parent variables is influenced by both the mean and variance estimates, which is consistent with the HNM formulation defined in the main paper (Eq. (3)), where the two networks share the same set of parents. In particular, to mitigate the failure mode in which the NLL loss and the DAG constraint interact harmfully in the early stages under the HNM assumption, the surrogate loss is designed to decouple the NLL into a mean term and a variance term, enabling stable joint optimization without significantly increasing computational cost.

A.4 Heteroscedastic Noise Vs. Heterogeneous Data

Data observed in the real world often violate the equal-variance assumption, and various studies in causal discovery address data heterogeneity. To avoid confusion, we distinguish the heteroscedastic noise we focus on from heterogeneous data. Heteroscedastic noise means the noise variance depends on the parent variable (cause) [47]. That is, the noise variance changes with the value of the input (cause), and consequently the variance differs across observations. By contrast, heterogeneous data refers to data in which the noise variables' distributions differ—typically distribution shifts across environments in multi-domain causal discovery [15, 26, 44, 52, 56]. This includes cases where the noise variance is 1 in environment A and 4 in environment B, or where variances differ by variable within the same environment. In short, heteroscedastic noise means variance depends on the cause and thus also differs at the observation level, whereas heterogeneous data means different noise types and variances across variables and environments. This distinction also appears in prior work: DARING [19] explicitly considers variables with heterogeneous noises; ReScore [52] proposes a method for multi-domain and multi-group settings, consistent with MC [15], CD-NOD [20], DICD [44], and FedCDH [26]. MEE [56] assumes noise distributions can differ both across and within environments, tackling a more challenging setting than prior work. We compare against these on HNM data; results appear in Appendix H.6.

B Details of Our Surrogate loss

B.1 Convergence Analysis

To theoretically analyze the convergence characteristics of the surrogate loss employed as a substitute for the NLL in the early stage of training, we make the following assumption about the scheduling coefficient $\lambda_{\text{reg}}(t)$ that controls the influence of the surrogate loss:

ASSUMPTION 1 (SMOOTHNESS OF SCHEDULING COEFFICIENT). *The scheduling function $\lambda_{\text{reg}}(t)$, which is continuously differentiable over the interval $[0, t^*]$ is assumed to satisfy the following Lipschitz continuity condition:*

$$|\lambda_{\text{reg}}(t + \Delta t) - \lambda_{\text{reg}}(t)| \leq L_\lambda \Delta t, \quad \forall t, \Delta t > 0, \quad (17)$$

where $L_\lambda > 0$ denotes a Lipschitz constant. We assume $\lambda_{\text{reg}}(t) \rightarrow 0$ as $t \rightarrow t^*$, which implies that the influence of the surrogate loss diminishes as training progresses. Consequently, the overall training objective smoothly transitions from one based on the surrogate loss to the standard NLL objective.

The gradual transition between losses induced by $\lambda_{\text{reg}}(t)$ prevents abrupt changes in the optimization landscape, ensuring that the solution at each training step does not drastically deviate from the solution reached in the previous step. It thus helps the optimizer converge in a consistent direction. This behavior is similar to the locally linear tracking condition of the homotopy-SGD proposed in Gargiani et al. [14].

B.1.1 Monotonicity of loss for smooth objective transition. We analyze the stability of the convergence path of the joint parameter sequence $\{(\Theta_t, W_t)\}$ depending on the objective transition. We use a quasi-Newton optimization algorithm such as L-BFGS, which chooses the step size at each iteration via a line search [45] satisfying the Wolfe conditions. As a result, the loss function decreases at every iteration, and if the loss is bounded below, the following monotonically decreasing condition holds:

$$\mathcal{L}_{\text{surrogate}}(\Theta^{t+1}, W^{t+1}, t+1) < \mathcal{L}_{\text{surrogate}}(\Theta^t, W^t, t). \quad (18)$$

The function $\mathcal{L}_{\text{surrogate}}$ is continuous w.r.t the parameter sequence and has a lower bound, so it converges under the above condition. Additionally, since $\lambda_{\text{reg}}(t)$ is Lipschitz continuous on the interval $[0, t^*]$ and $\lambda_{\text{reg}}(t) \rightarrow 0$ as $t \rightarrow t^*$, the rate of change of the loss over time gradually decreases as training proceeds. As a result, the form of the overall objective function changes gradually: it starts from a simple form dominated by the surrogate loss and slowly transitions to the form of NLL with the acyclicity constraint. This gradual objective transition suppresses abrupt oscillations in the optimization trajectory and increases the stability of the convergence path. According to the analysis in Deng et al. [8], such a gradual transition can lead to a stationary trajectory. Although our setting is the general non-linear, multivariate setting (more complex than the bivariate-linear setting studied in Deng et al. [8]), their observation that slowly decaying the coefficient leads to a smooth change in trajectory direction and preserves the continuity of the convergence trajectory can similarly apply to our proposed method.

B.1.2 Convergence to a stationary point. To show that the joint parameter sequence $\{(\Theta^t, W^t)\}$ converges to a stationary point set, following Yin et al. [48], we apply the convergence theorem of Wu [46]. According to this theorem, convergence is guaranteed if the following two conditions are satisfied: (i) The parameter update mapping $\mathcal{M}$ is a closed point-to-set map on the complement of the stationary set $\mathcal{F}$, and (ii) If $(\Theta^{t-1}, W^{t-1}) \notin \mathcal{F}$, then the loss decreases at iteration t , i.e., $\mathcal{L}_{\text{surrogate}}(\Theta^t, W^t, t) < \mathcal{L}_{\text{surrogate}}(\Theta^{t-1}, W^{t-1}, t-1)$, and thus the joint parameter sequence satisfies the monotonicity condition (Eq. (18)). Following our assumption, $\mathcal{L}_{\text{surrogate}}$ is continuous w.r.t the parameter sequence, and $\mathcal{M}$ (defined by the optimization trajectory) is a continuous point-to-set map, thus satisfying the closedness condition. Therefore, the joint parameter sequence $\{(\Theta^t, W^t)\}$ converges to the stationary set $\mathcal{F}$. Once training has progressed sufficiently, (Θ^t, W^t) will lie in a region where the gradient norm of the loss is below ε , i.e., within some ε -stationary point.

B.1.3 Remark. Once the early phase of training is over, the mean and variance networks have converged stably and W is optimized within a representation that explains the data well. In the later stage, as the influence of the scheduling coefficient diminishes, the NLL loss and the acyclicity constraint fully come into effect, thereby facilitating effective DAG learning. This indicates that the graduated optimization, which gradually adjusts the influence of each loss term, alleviates instabilities that may occur in the early stage of training and helps the overall training trajectory converge stably to a stationary point. To maintain the continuity of such a stationary trajectory, the stationary point at each step must be isolated and the corresponding Jacobian must be invertible. Deng et al. [8] theoretically showed that under this condition, the homotopy trajectory is preserved. Although a complete guarantee is difficult in our nonlinear and multivariate setting, the design of the scheduling coefficient $\lambda_{\text{reg}}(t)$ helps ensure that the trajectory continues smoothly without discontinuities or abrupt changes in the loss landscape.

B.2 Justification for Our Surrogate

The proposed surrogate loss $\mathcal{L}_{\text{surrogate}}$ is designed to decouple the components of the NLL loss: the mean function is trained with an MSE loss and the variance function is trained with a residual-based regularization loss. In this subsection, we aim to demonstrate that the proposed loss function indeed serves as a surrogate for the NLL by comparing the gradients of these two loss terms with the gradient of the NLL loss.

B.2.1 Gradient consistency. We decouple the mean and variance components of the NLL and compute the gradient for each. To this end, we first define the NLL loss for a single sample as follows:

$$\mathcal{L}_{\text{NLL}}^{(j)} = \frac{1}{2} \log \sigma_j^2 + \frac{(X_j - \mu_j)^2}{2\sigma_j^2}, \quad (19)$$

where $\mu_j = \mu_j(X_j; W_j; \theta_\mu)$ and $\sigma_j^2 = \sigma_j^2(X_j; W_j; \theta_\sigma)$ denote the predictive mean and variance, respectively. The gradients of $\mathcal{L}_{\text{NLL}}^{(j)}$ with respect to μ_j and σ_j^2 are:

$$\nabla_{\mu_j} \mathcal{L}_{\text{NLL}}^{(j)} = -\frac{X_j - \mu_j}{\sigma_j^2}, \quad \nabla_{\sigma_j^2} \mathcal{L}_{\text{NLL}}^{(j)} = \frac{1}{2\sigma_j^2} - \frac{(X_j - \mu_j)^2}{2\sigma_j^4}. \quad (20)$$

The stationary point at which the gradients with respect to the mean and variance are zero is given by:

$$\nabla_{\mu_j} \mathcal{L}_{\text{NLL}}^{(j)} = 0 \Rightarrow \mu_j = X_j, \quad (21)$$

$$\nabla_{\sigma_j^2} \mathcal{L}_{\text{NLL}}^{(j)} = 0 \Rightarrow \sigma_j^2 = (X_j - \mu_j)^2. \quad (22)$$

Based on the above stationary point, we construct the surrogate loss with the following two terms:

$$\mathcal{L}_{\text{MSE}}^{(j)} = (X_j - \mu_j)^2, \quad \mathcal{L}_{\text{VarReg}}^{(j)} = \left(\sigma_j^2 - (X_j - \mu_j)^2 \right)^2. \quad (23)$$

The gradient of each loss term is as follows:

$$\nabla_{\mu_j} \mathcal{L}_{\text{MSE}}^{(j)} = -2(X_j - \mu_j), \quad \nabla_{\sigma_j^2} \mathcal{L}_{\text{VarReg}}^{(j)} = 2 \left(\sigma_j^2 - (X_j - \mu_j)^2 \right). \quad (24)$$

Now, assuming the surrogate loss has reached a stationary point, the corresponding predictive mean and variance are given by:

$$\begin{aligned} \nabla_{\mu_j} \mathcal{L}_{\text{MSE}}^{(j)} = 0 &\Rightarrow \mu_j = X_j, \\ \nabla_{\sigma_j^2} \mathcal{L}_{\text{VarReg}}^{(j)} = 0 &\Rightarrow \sigma_j^2 = (X_j - \mu_j)^2. \end{aligned} \quad (25)$$

Substituting the above results into the gradient expressions of the NLL, we obtain:

$$\nabla_{\mu_j} \mathcal{L}_{\text{NLL}}^{(j)} = 0, \quad \nabla_{\sigma_j^2} \mathcal{L}_{\text{NLL}}^{(j)} = 0. \quad (26)$$

Thus, if the surrogate loss reaches a stationary point, the NLL loss is also at a stationary point. Meanwhile, as $\lambda_{\text{reg}}(t)$ decays over time and approaches zero, the influence of $\lambda_{\text{reg}}(t)$ in the surrogate loss vanishes and $\mathcal{L}_{\text{StopNLL}}$ becomes the dominant term. This term blocks gradients from propagating to the mean network, whereas for the variance network it operates in the same form as the NLL loss. Therefore, the gradient of the overall surrogate loss asymptotically aligns with the gradient of the NLL:

$$\|\nabla \mathcal{L}_{\text{surrogate}} - \nabla \mathcal{L}_{\text{NLL}}\| \rightarrow 0. \quad (27)$$

B.2.2 Analysis of curvature and condition number. We now analyze whether the surrogate loss is a better-conditioned function than the NLL loss by comparing the Hessians of each loss. First, for the NLL loss, the diagonal elements of the Hessian (i.e. the second derivatives) with respect to the mean and variance are as follows:

$$\nabla_{\mu_j}^2 \mathcal{L}_{\text{NLL}}^{(j)} = \frac{1}{\sigma_j^2}, \quad \nabla_{\sigma_j^2}^2 \mathcal{L}_{\text{NLL}}^{(j)} = -\frac{1}{2\sigma_j^4} + \frac{(X_j - \mu_j)^2}{\sigma_j^6}. \quad (28)$$

The above expressions indicate that as σ_j^2 becomes smaller, the magnitude of the second derivative may increase sharply, which can cause ill-conditioning; on the other hand, as σ_j^2 becomes larger, the magnitude of the second derivative can become very small, potentially causing the training to stall. This issue is particularly problematic in joint optimization, where there is a tendency for the variance to increase in order to reduce the NLL in the early stage of training.

In contrast, the second derivatives of each term composing the surrogate loss are as follows:

$$\nabla_{\mu_j}^2 \mathcal{L}_{\text{MSE}}^{(j)} = 2, \quad (29)$$

$$\nabla_{\mu_j}^2 \mathcal{L}_{\text{VarReg}}^{(j)} = 12(X_j - \mu_j)^2 - 4\sigma_j^2 = 4(3(X_j - \mu_j)^2 - \sigma_j^2),$$

$$\nabla_{\sigma_j^2}^2 \mathcal{L}_{\text{VarReg}}^{(j)} = 2.$$

- For μ_j , $\mathcal{L}_{\text{MSE}}^{(j)}$ is a quadratic function and is locally more convex than the NLL, with its second derivative always constant at 2. This means that the loss landscape in the μ_j direction has uniform curvature, enabling more stable gradient updates during the optimization process.
- Meanwhile, although the second derivative of $\mathcal{L}_{\text{VarReg}}^{(j)}$ does vary depending on the difference between X_j and μ_j , it exhibits relatively mild curvature with respect to changes in σ_j^2 . In particular, even as $\sigma_j^2 \rightarrow 0$, this curvature does not diverge; and even if σ_j^2 becomes very large, the curvature does not approach 0, leading to a smooth training in both extremes.

- For σ_j^2 , the second derivative of $\mathcal{L}_{\text{VarReg}}^{(j)}$ in the direction of σ_j^2 is constant at 2. Since the curvature in the σ_j^2 direction is constant, the loss landscape remains stable without excessive oscillation even when the predictive variance is very small in the early training stage. Moreover, the gradually applied $\mathcal{L}_{\text{StopNLL}}$ term prevents the unstable variance gradient in the early stages from affecting the mean trajectory, as the gradient with respect to μ_j does not propagate.

As a result, compared to the NLL loss, the surrogate loss avoids sudden curvature changes and numerical instabilities caused by high-order variance terms (e.g. $1/\sigma_j^4$, $1/\sigma_j^6$), and yields an optimization landscape with a lower condition number. This can be summarized by the inequality:

$$\kappa(\nabla^2 \mathcal{L}_{\text{surrogate}}) < \kappa(\nabla^2 \mathcal{L}_{\text{NLL}}), \quad (30)$$

where $\kappa(\cdot)$ denotes the condition number of the corresponding Hessian.

B.2.3 Toy example: Heteroscedastic regression. Stirn et al. [41] demonstrated the potential for stable convergence of nonlinear models by using the stop-gradient operation to decouple the updates of the mean and variance networks so that they proceed independently. Similarly, our proposed $\mathcal{L}_{\text{surrogate}}$ decouples the mean and variance contributions of the NLL loss, minimizing their interference with each other and enabling a balanced optimization. In addition, the scheduling coefficient $\lambda_{\text{reg}}(t)$ assigns a large weight in the early stage of training to induce fast convergence, and gradually allows the objective function to transition smoothly to the NLL loss; this operates on a principle similar to the *temperature annealing* proposed in Upadhyay et al. [43].

To verify that optimization under our $\mathcal{L}_{\text{surrogate}}$ is more stable than under the NLL loss, we conducted an experiment following Seitzer et al. [34]. In this experiment, we consider a heteroscedastic regression problem of simultaneously predicting the mean and variance for data generated from a sine function with heteroscedastic noise. We trained a two-layer MLP with sigmoid activations on this data. Figure 6 visualizes the results of this experiment. It shows that when using $\mathcal{L}_{\text{surrogate}}$, the predictive mean aligns with the true curve of the input data more closely than when using the NLL. For a quantitative evaluation metric we used the uncertainty calibration error (UCE), which measures how accurately the predictive variance explains the residuals. The results showed that the proposed method achieved a lower UCE compared to NLL, indicating that it effectively estimates the uncertainty (variance) of data while also fitting the mean well. These findings empirically support that $\mathcal{L}_{\text{surrogate}}$ can converge more stably than the NLL.

C Details of Graduated Optimization

Our proposed objective transition approach based on the weighted loss scheduling guides the model parameters to converge stably in the early stage of training. Until the predictive mean fits the data well, it suppresses the influence of the complex NLL loss and ensures that W is updated in a direction that explains the data effectively. This in turn effectively mitigates the phenomenon in which an incorrect mean estimate distorts both the predictive variance and the DAG learning process.

Conceptually, our optimization strategy is closely related to homotopy optimization and graduated non-convexity (GNC) techniques. Homotopy optimization [8, 14] sets a simple, easy-to-optimize surrogate objective at the beginning and then gradually transitions to the original complex objective function to find the optimal solution. GNC is likewise an approach that, rather than directly optimizing a nonconvex objective function, gradually transforms a convex or otherwise smooth surrogate loss into a progressively less flat form during training [18, 25]. These methods share the following characteristics: (i) they do not directly minimize a non-convex function with many local minima; (ii) they begin with a smooth and well-conditioned loss function; and (iii) by gradually transitioning the objective function to the nonconvex form, they maintain the prior optimization trajectory. Our proposed weighted loss scheduling optimization follows the same principles. Initially, training is performed using loss functions that are smoother than the NLL (such as $\mathcal{L}_{\text{MSE}}$ and $\mathcal{L}_{\text{VarReg}}$), and subsequently the objective function is gradually transitioned to the NLL loss and the acyclicity constraint. This process alleviates instabilities caused by abrupt changes in the gradient surface. Therefore, this work can be regarded as the first example of applying the classical concepts of GNC and homotopy to DAG learning under HNMs.

We state the assumptions used to derive conditions on the hyperparameters.

ASSUMPTION 2 (INITIAL CONSTRAINT SUPPRESSION). *The penalty coefficient ρ for the ALM is increased gradually, remaining small during the initial training phase ($t \leq T_1$). This approach prevents the acyclicity constraint from prematurely dominating the learning process, allowing the data reconstruction signal to guide the initial updates of the DAG structure.*

ASSUMPTION 3 (VARIANCE ALIGNMENT). *During the initial phase where the scheduling coefficient $\lambda_{\text{reg}}(t)$ is large, the surrogate loss terms, particularly $\mathcal{L}_{\text{VarReg}}$ and $\mathcal{L}_{\text{StopNLL}}$, are designed to align the predicted variance σ^2 with the magnitude of the empirical residuals. This mechanism calibrates the variance predictor and suppresses its tendency to overestimate, which is a common issue in joint optimization with NLL loss.*

ASSUMPTION 4 (SLOW TRANSITION). *The scheduling coefficient $\lambda_{\text{reg}}(t)$ must decay slowly. A gradual transition from the surrogate objective to the standard NLL is critical for maintaining the continuity of the homotopy path, ensuring that the optimization trajectory does not experience abrupt shifts that could lead to suboptimal solutions.*

C.1 Analysis of Conditions for Schedule Parameters

As suggested in [8], homotopy optimization requires three conditions to maintain a continuous stationary trajectory toward the global optimum; we especially focused on the second condition, which stipulates that the coefficient $\lambda_{\text{reg}}(t)$ must change only very slowly. The remaining conditions (the stationary point must be isolated and the Jacobian around the solution must be invertible)

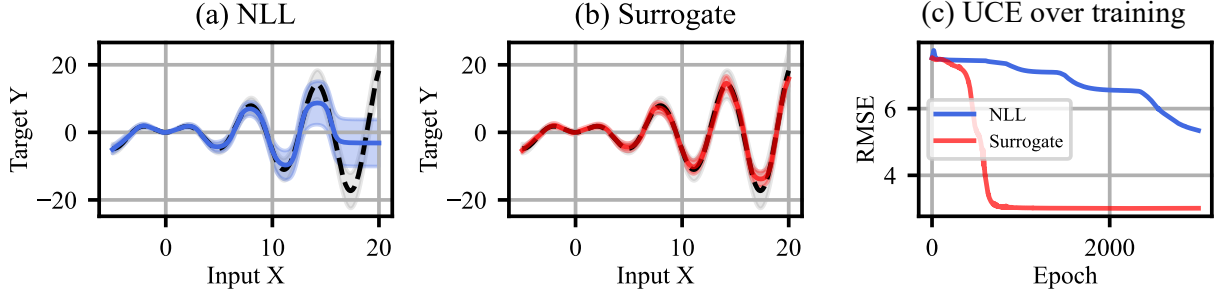


Figure 6: Fits for the heteroscedastic sine example from [34]. A sine curve with increasing amplitude and noise: $y(x) = x\sin(x) + x\xi_1 + \xi_2$ where ξ_1 and ξ_2 are Gaussian noise with standard deviation $\sigma = 0.3$. Uncertainty Calibration Error (UCE) is used as an evaluation metric.

depend largely on the model architecture and the optimizer’s characteristics, so we do not consider those two here. In relation to this second condition, the adjustable factors in our method are the schedule parameters $\lambda_{\text{reg}}(0)$ and t^* . $\lambda_{\text{reg}}(0)$ determines the relative weighting between the reconstruction loss and the acyclicity constraint $h(W)$ in the initial objective function. If $\lambda_{\text{reg}}(0)$ is small, both terms are incorporated early on in a balanced manner, whereas if it is large, the reconstruction loss dominates the initial optimization. t^* represents the transition point (but it is also related to the decay rate τ). The hyperparameter analysis in Section 4.3.4 of the main paper presents experiments on τ . If t^* is very small, the penalty term increases abruptly and the loss landscape changes quickly, potentially violating the condition that the coefficient $\lambda_{\text{reg}}(t)$ should decay slowly. Conversely, if t^* is excessively large, the acyclicity constraint may take effect too late, causing the optimization to remain in a suboptimal regime.

We conducted DAG learning experiments under various settings to examine how changes in the schedule parameters affect training dynamics and performance. The experimental setup and procedure were identical to those described in Section 2.2 and Figure 2 of the main paper. Specifically, we generated 10 different synthetic datasets with $d = 10$ under the ER1 condition, and for each we evaluated the contribution ratio of the two loss functions, the value of $h(W)$, and the SHD (Structural Hamming Distance). Figures 7 and 8 show the results depending on varying $\lambda_{\text{reg}}(0)$. When $\lambda_{\text{reg}}(0)$ is small, the contribution ratio quickly approached 1 from the very beginning of training, and $h(W)$ also decreased rapidly; SHD initially decreased and then tended to stagnate at a somewhat high value. In contrast, as $\lambda_{\text{reg}}(0)$ becomes larger (e.g., more than 150), $h(W)$ decreases slowly at first and then exhibits abnormal behavior such as rising again or oscillating, and in some cases the contribution ratio exceeded 1. The SHD values also showed an unstable pattern of decreasing, then increasing, then decreasing again. Overall, the range $100 \leq \lambda_{\text{reg}}(0) \leq 130$ results in relatively stable training dynamics.

Figures 9 and 10 present the results depending on different t^* settings. When t^* is very small (e.g., 50 to 300), the contribution ratio fluctuated irregularly and $h(W)$ dropped quickly at first but then its value oscillated. On the other hand, for values $t^* \geq 1000$, the contribution ratio remained stable and $h(W)$ decreased in a smooth, natural manner. Notably, beyond about $t^* \geq 1300$, we

observed in some cases that early in training the contribution ratio would spike or $h(W)$ would actually increase. In conclusion, the range $1000 \leq t^* \leq 1200$ appears to be the most effective for DAG learning, as the SHD was gradually reduced within this range.

These results suggest that by appropriately tuning the schedule parameters, one can empirically satisfy the condition from Deng et al. [8] that the coefficient changes only slowly. In other words, even in a complex nonlinear DAG learning setting, choosing suitable schedule parameters can maintain stable optimization, demonstrating that the theoretical condition does have a certain degree of effectiveness in practical optimization.

C.2 Trajectory Analysis

We summarize the theoretical context of the proposed methodology through visualizations. The proposed method shares a similar objective with the spirit of homotopy optimization [1, 14] and the graduated non-convexity (GNC) method [18, 21]. Both methods share the characteristic that, in early training stages, they use an objective function that is easy to optimize and then gradually transform it continuously into the original nonlinear, nonconvex problem. The homotopy method proceeds along an “easy-to-solve” path, and it is classically known that following this path yields a stationary path [1]. GNC similarly operates on the same principle: it starts with smoothing a nonconvex function using a smoothing parameter, and then gradually reduces the level of flatness, allowing the optimizer to avoid poor local minima and gradually converge to better solutions [18]. Our method can be connected to homotopy’s continuous path-tracking effect by gradually adjusting the relative influence between loss terms via $\lambda_{\text{reg}}(t)$ and by using a surrogate loss for the NLL in the early stage.

In this connection, Deng et al. [8] theoretically presented conditions under which, for a linear bivariate DAG learning problem, a stationary path can be structurally maintained if the penalty is increased gradually. This explains that if the stationary point is isolated, the Jacobian around the solution is invertible, and the penalty is increased sufficiently slowly, then the entire optimization path can continue without breaks. However, such conditions generally do not hold in nonlinear multivariate structure learning involving MLPs, and in particular, due to the nonconvex nature of the loss landscape, even if a stationary path exists, there is no guarantee that it will be continuous. Nonetheless, we empirically observed

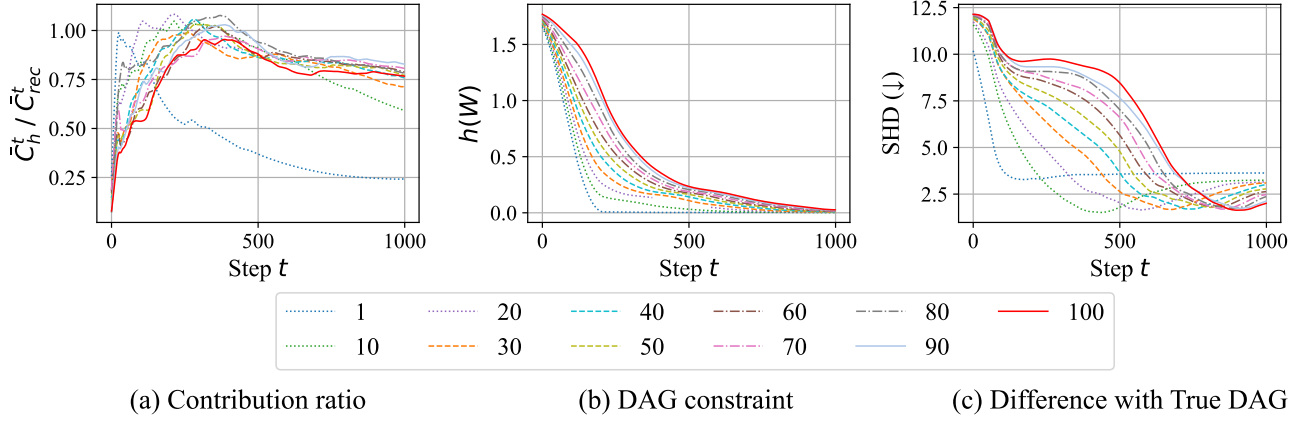


Figure 7: Analysis on initial training dynamics using small $\lambda_{\text{reg}}(0)$ values (within $\{1, 10, \dots, 100\}$).

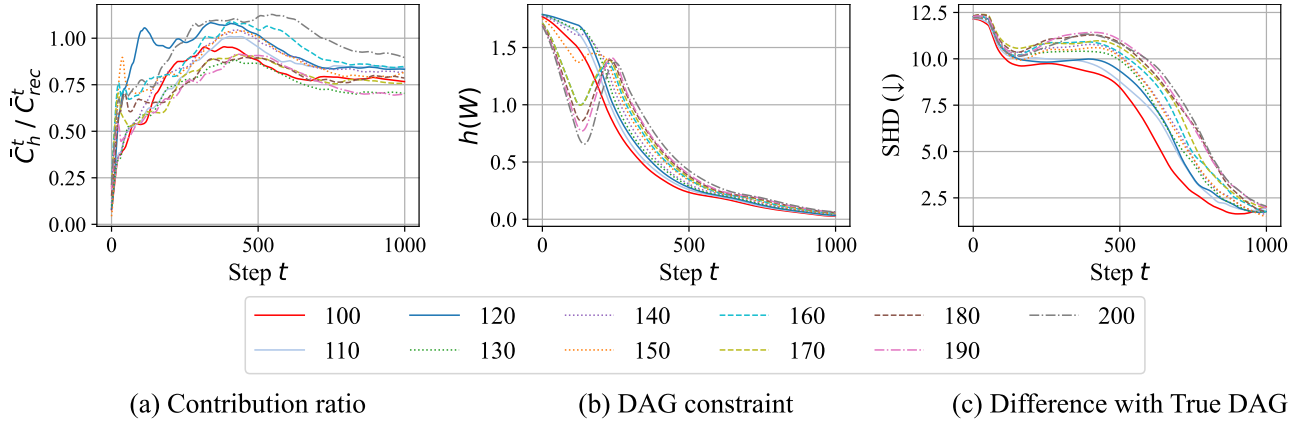


Figure 8: Analysis on initial training dynamics using large $\lambda_{\text{reg}}(0)$ values (within $\{100, 110, \dots, 200\}$).

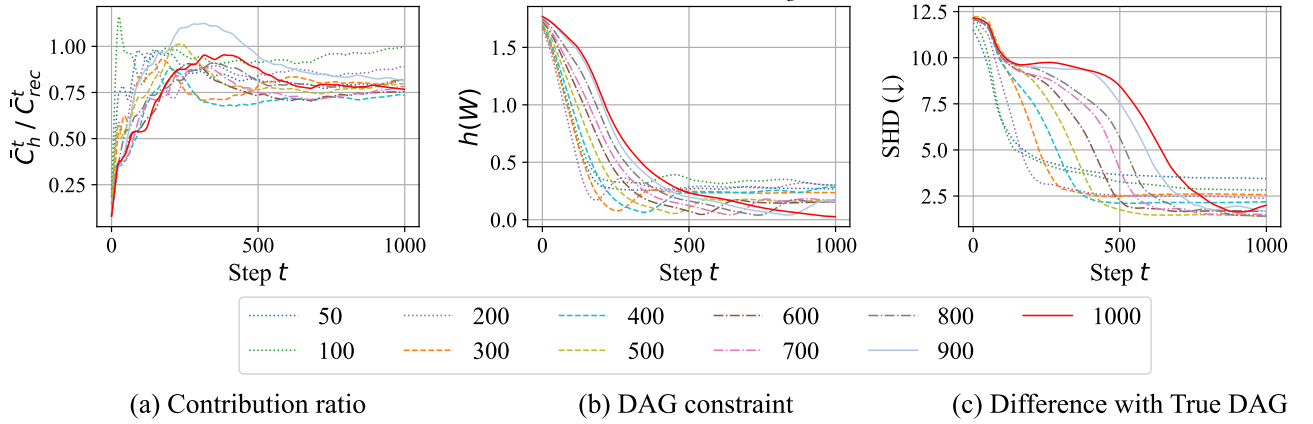


Figure 9: Analysis on initial training dynamics using small t^* values (within $\{50, 100, 200, \dots, 1000\}$).

the influence of the proposed optimization strategy on the actual training trajectory.

Figure 11 compares the loss contours and optimization trajectories visualized via PCA under different conditions for the ER1 graph with $d = 10$ (see Appendix E.1 for its detailed settings). The proposed method’s trajectory is very smooth overall, and the loss

contours near the convergence point form nearly concentric circles. Compared to the proposed method (a), all alternative settings (b–d) exhibit more unstable optimization behaviors. In particular, the highly reduced surrogate influence (c) produces a zigzag trajectory, while a rapid transition to NLL (d) results in a distorted loss landscape with sharp or elongated contours. Meanwhile, abrupt

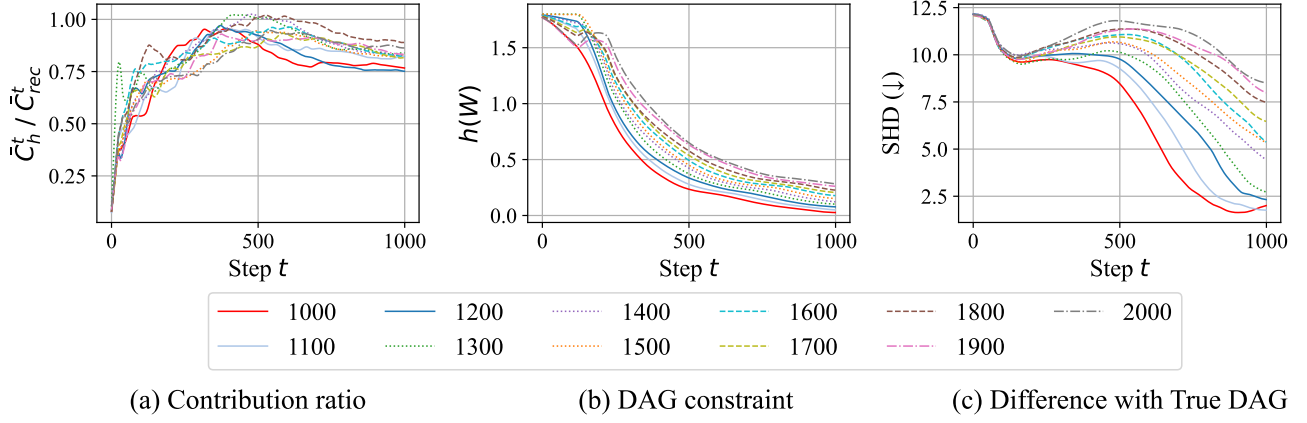


Figure 10: Analysis on initial training dynamics using large t^* values (within $\{1000, 1100, \dots, 2000\}$).

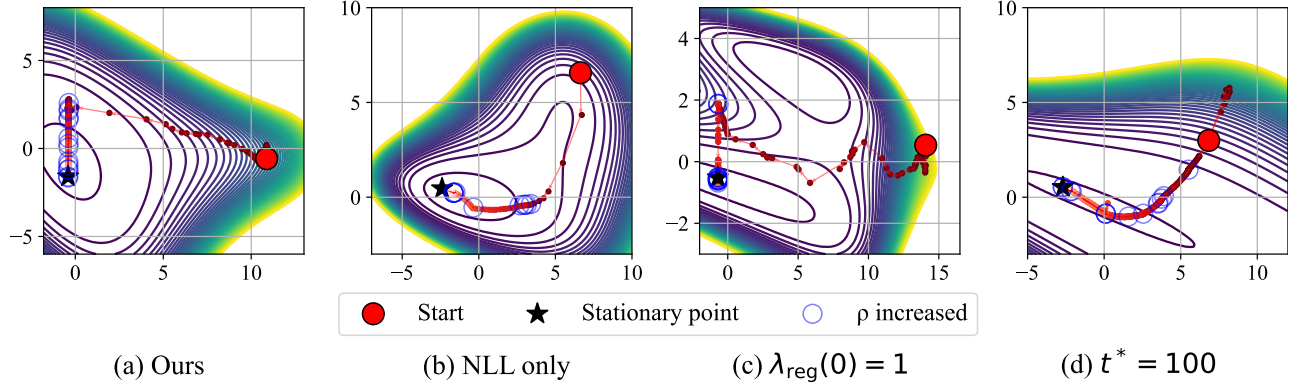


Figure 11: Trajectory analysis of the optimization. Contours closer to concentric circles indicate a more stable loss landscape, and trajectories closer to straight lines imply more stable convergence.

directional changes in the trajectory are observed across all settings (a–d), which aligns with prior findings that such instability can naturally occur in ALM-based L-BFGS optimization [8, 30]. Importantly, in our method, even after this change in direction, the trajectory continues smoothly. This suggests that the loss scheduling coefficient was designed to be neither too fast nor too slow, allowing the optimization at each stage to stay well within the attraction basin of the next stationary path. Even though our experimental setting involves a multivariate structure and nonlinear models that do not strictly satisfy the mathematical assumptions of Deng et al. [8], the fact that the overall path continues without interruption and that increasing the penalty does not cause discontinuous changes in the trajectory provides empirical support for the theory.

To intuitively compare the influence of the schedule parameters on the optimization trajectory, we projected each trajectory obtained from different parameter values into the same two-dimensional space. Figure 12 shows the visualization, where both trajectories in each of (a) and (b) were projected onto the same PCA space, aligning their starting points to the same location.

Figure 12(a) shows the differences in trajectory when varying $\lambda_{reg}(0)$. When $\lambda_{reg}(0)$ is small (shown in green), the effect of the penalty term is manifested early, causing ρ to increase rapidly and inducing premature convergence. In this case, the trajectory

explores only a narrow region of the loss landscape, which may increase the risk of getting trapped in local minima or saddle points. In contrast, when $\lambda_{reg}(0)$ is large (red), the reconstruction loss dominates the initial optimization and ρ increases only later. As a result, the trajectory covers a substantially broader region of the loss landscape, and convergence begins after sufficient exploration once ρ eventually starts to rise. This tendency allows for a better understanding of the geometry of the loss landscape and offers a high potential for convergence to a better stationary point.

Figure 12(b) illustrates how the trajectory changes with different values of t^* . When it is large (red), the influence of the penalty term grows gradually, leading the optimization to proceed stably; a clear convergence trend appears when ρ eventually increases at the final stage. Although the trajectory exhibits some wobbling initially, the optimization followed a consistent desired direction until the penalty was strengthened. In contrast, when t^* is small (green), the penalty’s influence comes into play early, causing the loss landscape to shift abruptly; consequently, the trajectory made discontinuous jumps or sharp turns and showed very unstable behavior.

As a result, we can see how the schedule parameters affect the dynamics and stability of the optimization trajectory; changes in $\lambda_{reg}(0)$ and t^* led to clear differences in the traversed region of the loss landscape, the point of convergence, and the consistency of

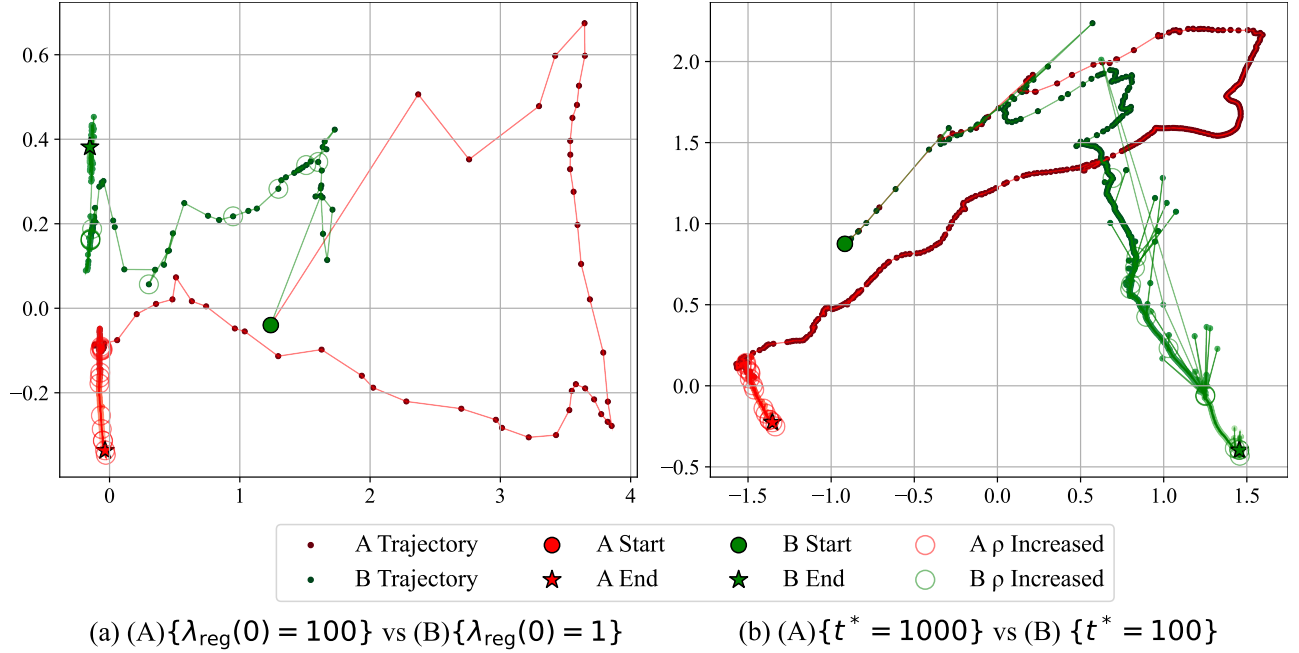


Figure 12: Visualization of optimization trajectories projected onto the same two-dimensional space. For fair comparison, we did not use all training steps, but instead adjusted the sampling interval of trajectory points to match the number of steps in the shorter trajectory of the two methods. Through this analysis, we aim to assess how wide the space exploration is.

the update direction. This suggests that when and how the penalty term dominates plays a critical role in maintaining a stable optimization trajectory. A gradual decrease of the scheduling coefficient directly impacts the continuity of the optimization trajectory and the accuracy of the learned DAG.

D Details for Section 3.2 of the Main Paper

In this section, we define the key notation required to substantiate the theoretical analysis developed in Section 3.2 of the main paper, in particular to prove how the weighted loss scheduling strategy regulates the balance between the reconstruction loss and the DAG constraint to prevent training failures.

D.1 Notations

First, we define the gradients representing the two key factors we aim to analyze. The update of W is determined by the interaction between the reconstruction loss and the DAG constraint. The reconstruction gradient $g_{\text{rec}}(t) = \nabla_W L_{\text{rec}}(t)$ is the loss signal indicating how well the current model explains the data. The DAG constraint gradient $g_h(t) = \nabla_W (\frac{\rho}{2} h(W_t)^2 + \alpha h(W_t))$ is the loss signal indicating how well the weight matrix W_t satisfies the DAG structure.

In the optimization process (using an L-BFGS optimizer), the change ΔW_t in the parameter W at each step t can be approximated as:

$$\Delta W_t = -\eta_t H_t (g_{\text{rec}}(t) + g_h(t)), \quad (31)$$

where η_t is the step size and $H_t \succeq 0$ is the L-BFGS inverse-Hessian approximation (a positive semi-definite matrix). This equation shows

that the update of W is a linear combination of the two gradients. As discussed in Section 3.2 of the main paper, it is important to quantitatively determine which gradient dominates the update of W in the early stage of training. The per-step contribution ratio Q_t indicates how much more strongly $g_h(t)$ influences the update of W compared to $g_{\text{rec}}(t)$ in a single step.

$$Q_t := \frac{|\langle H_t(g_{\text{rec}}(t) + g_h(t)), g_h(t) \rangle|}{|\langle H_t(g_{\text{rec}}(t) + g_h(t)), g_{\text{rec}}(t) \rangle|}. \quad (32)$$

The cumulative contribution ratio is defined as the average of the per-step contribution ratios up to time T_0 (this is the metric used in Definition 3 of the main paper).

$$\frac{C_h(T_0)}{C_{\text{rec}}(T_0)} = \frac{1}{T_0} \sum_{t=0}^{T_0-1} Q_t. \quad (33)$$

If this ratio is greater than 1, it means that over that period the constraint term dominated the learning more than the reconstruction loss did. Finally, let κ denote the condition number of the Hessian $\nabla_W^2 \mathcal{L}_t$ of the loss function, which indicates the stability of the optimization problem. A large value of κ implies that the loss surface is unstable and optimization can become difficult.

$$\kappa := \text{cond}(\nabla_W^2 \mathcal{L}_t) = \frac{\lambda_{\max}(\nabla_W^2 \mathcal{L}_t)}{\lambda_{\min}(\nabla_W^2 \mathcal{L}_t)} \in [1, \infty). \quad (34)$$

D.2 Failure Certificate

When optimizing the standard objective (Eq. (3) of the main paper) under an HNM, the training can sometimes converge prematurely

in the wrong direction. The failure certificate is a concept that formally defines and diagnoses this optimization failure mechanism. It quantitatively captures the phenomenon where, under certain conditions, the learned DAG structure is determined too hastily by the acyclicity constraint rather than by the true data relationships. A failure certificate is defined as the scenario in the early training interval $[0, T_0]$ where the following two conditions hold simultaneously: constraint-dominant and premature acyclicity. Here, constraint-dominant means that the cumulative contribution of the constraint relative to the reconstruction loss stays above some constant $\gamma > 1$ (indicating that the optimization is overly focused on enforcing acyclicity — essentially, the model is forming an acyclic graph without sufficiently learning the data's patterns). Premature acyclicity means that by the early time T_0 , the value of $h(W)$ has converged below an extremely small threshold ϵ , indicating a premature lock-in phenomenon wherein the graph structure has been finalized before the model has adequately learned from the data.

$$\frac{C_h(t)}{C_{\text{rec}}(t)} \geq \gamma \quad (0 \leq t \leq T_0), \quad h(W_{T_0}) \leq \epsilon. \quad (35)$$

This failure arises from the inherent instability of the NLL loss under an HNM. Early in training, the variance tends to be overestimated, and because the reconstruction component of the NLL loss is inversely proportional to this variance, its influence is greatly attenuated. The DAG constraint, which grows increasingly stronger due to the ALM, can then easily overwhelm the reconstruction loss, thereby satisfying the conditions of the failure certificate described above. Moreover, this phenomenon occurred even with β -NLL (which was designed to mitigate the NLL loss's instability), and we regard this issue as stemming from the model architecture under HNM — that is, a network that must output both a mean and a variance. This demonstration of failure serves as a key motivation for our proposed progressive optimization strategy based on weighted loss scheduling. Our aim is to deliberately strengthen the influence of the reconstruction loss in the early stage of training so as to prevent these failure conditions from occurring, and to gradually induce the formation of the DAG structure only after the model has sufficiently learned from the data.

D.3 Auxiliary Inequalities

In the next section, we present two key inequalities needed to complete the proof of Lemma 3.1 stated in the main paper. These auxiliary results yield an upper bound on the per-step contribution ratio Q_t and analyze conditions under which this bound can be controlled.

LEMMA D.1 (QUADRATIC-FORM BOUND FOR Q_t). *First, to obtain a more tractable expression for Q_t at each step, we define an upper bound as a function of u , which encodes the relative magnitude of the reconstruction gradient g_{rec} and the constraint gradient g_h .*

Let $a := \langle H_t g_{\text{rec}}, g_{\text{rec}} \rangle$, $b := \langle H_t g_h, g_h \rangle$, $c := \langle H_t g_{\text{rec}}, g_h \rangle$, and $u := \sqrt{b/a} \in [0, 1]$. Then

$$Q_t \leq \frac{b + \sqrt{ab}}{a - \sqrt{ab}} = u \frac{u+1}{1-u} =: \phi(u). \quad (36)$$

PROOF. Triangle inequality and Cauchy–Schwarz in the H_t -inner product yield $|\langle H_t (g_{\text{rec}} + g_h), g_h \rangle| \leq b + \sqrt{ab}$ and $|\langle H_t (g_{\text{rec}} + g_h), g_{\text{rec}} \rangle| \geq a - \sqrt{ab}$, giving Eq. (36). $\square$

LEMMA D.2 (MONOTONICITY AND THRESHOLD). *Next, we analyze properties of the upper-bound function $\phi(u)$ defined in Lemma 3.1 stated in the main paper. By exploiting its monotonicity, we identify a clear threshold u_δ such as Q_t remains below a target value $1 - \delta$. On $[0, 1]$, $\phi(u) = u(u+1)/(1-u)$ is strictly increasing with $\phi(0) = 0$. Let $u_\delta \in (0, 1)$ solve $\phi(u_\delta) = 1 - \delta$, i.e.,*

$$\frac{u_\delta(u_\delta + 1)}{1 - u_\delta} = 1 - \delta \implies u_\delta = \frac{-(2 - \delta) + \sqrt{(2 - \delta)^2 + 4(1 - \delta)}}{2}. \quad (37)$$

Then $u \leq u_\delta \implies Q_t \leq 1 - \delta$.

PROOF. Direct calculus and inversion of a quadratic. $\square$

Using these two auxiliary lemmas, we show that if the relative gradient size u stays below the threshold u_δ the per-step contribution ratio Q_t can be controlled within the desired range—this plays a central role in the final proof presented next.

D.4 Proof of Lemma 3.1 stated in the Main Paper

Fix $\delta \in (0, 1)$ and $T_1 \in \mathbb{N}$. If for all $t \leq T_0$,

$$\lambda_{\text{reg}}(t) \left\| \nabla_W (\mathcal{L}_{\text{MSE}} + \mathcal{L}_{\text{VarReg}}) \right\| \geq \frac{\kappa}{1 - \delta} \left\| \nabla_W \left(\frac{\rho}{2} h(W)^2 + \alpha h(W) \right) \right\|, \quad (38)$$

then $Q_t \leq 1 - \delta$ for all $t \leq T_0$, hence $\frac{C_h(T_0)}{C_{\text{rec}}(T_0)} \leq 1 - \delta$.

PROOF. By spectral bounds of H_t ,

$$u = \sqrt{\frac{\langle H_t g_h, g_h \rangle}{\langle H_t g_{\text{rec}}, g_{\text{rec}} \rangle}} \leq \sqrt{\kappa} \frac{\|g_h\|}{\|g_{\text{rec}}\|}. \quad (39)$$

In the early phase with stop-gradient,

$$g_{\text{rec}}(t) = \lambda_{\text{reg}}(t) \nabla_W (\mathcal{L}_{\text{MSE}} + \mathcal{L}_{\text{VarReg}}), \quad \nabla_W \mathcal{L}_{\text{StopNLL}}(t) = 0. \quad (40)$$

With stop-gradient Eq. (40), assumption Eq. (38) implies $\|g_{\text{rec}}\| \geq \frac{\kappa}{1 - \delta} \|g_h\|$, hence

$$u \leq \frac{1 - \delta}{\sqrt{\kappa}} < 1. \quad (41)$$

Combining Lemma D.1 and D.2, $u \leq u_\delta$ yields $Q_t \leq 1 - \delta$. A sharp sufficient condition is

$$\|g_{\text{rec}}\| \geq \frac{\sqrt{\kappa}}{u_\delta} \|g_h\|, \quad (42)$$

which is implied by Eq. (38) whenever $\kappa \geq \kappa_\delta := (\frac{1 - \delta}{u_\delta})^2$. Averaging over $t \leq T_0$ and using Eq. (33) gives the claim. $\square$

D.5 Path Stability under Slow Scheduling

This section provides the theoretical rationale for why, in the proposed weighted-loss scheduling, it is important to decrease the scheduling coefficient $\lambda_{\text{reg}}(t)$ slowly. This principle ensures that the optimization does not drift toward unstable regions but instead follows a stable homotopy path to a convergent solution.

To guarantee stable path tracking, we first assume that the scheduling coefficient $\lambda_{\text{reg}}(t)$ changes only slightly at each step (i.e., *decays slowly*):

$$|\lambda_{\text{reg}}(t+1) - \lambda_{\text{reg}}(t)| \leq L_\lambda/t^*, \quad (43)$$

We also assume that the gradient of the loss does not change abruptly ($(W, \lambda) \mapsto \nabla_W \mathcal{L}_{\text{sur}}(W; \lambda)$ is L -Lipschitz), and that around the solution path induced by the data the loss has a locally unique minimizer (local strong convexity (or PL-type) holds along the data-aligned stationary path with smallest Hessian eigenvalue $\mu > 0$). When these conditions are satisfied, we can apply the implicit function theorem, which guarantees that if the problem parameter (λ_{reg}) changes slightly, then the solution changes only slightly as well. Then by the implicit function theorem [1, 18],

$$\|W^*(\lambda') - W^*(\lambda)\| \leq \frac{L}{\mu} |\lambda' - \lambda|, \quad (44)$$

In conclusion, this bound implies that if the scheduling coefficient $\lambda_{\text{reg}}(t)$ changes slowly enough, the optimal solution W^* at each step also moves only gradually. Therefore, our optimization algorithm can stably track this slowly moving target (W^*). This prevents the optimization path from suddenly jumping in a different direction or reverting to the previously discussed constraint-dominant state, ultimately helping it converge to a better solution.

D.6 More details of Claim 1 stated in the Main Paper

This section provides a detailed proof of Claim 1 stated in the main paper. The core of Claim 1 stated in the main paper is that the proposed scheduling strategy avoids the failure certificate into which standard NLL optimization falls, thereby learning a more accurate DAG structure with the same number of samples ($\xi_{\text{ours}} < \xi_{\text{NLL}}$).

D.6.1 Assumptions. We impose the following four standard assumptions for the proof.

(i) **Uniform convergence:** The empirical risk computed from a finite sample converges to the population risk when the sample size is sufficiently large.

$$\sup_W |r_{\rho,n}(W) - r_\rho(W)| \leq \Delta_n \quad (45)$$

(ii) **Local PL condition:** In a neighborhood of the data-induced optimal solution path ($\mathcal{N}$), the loss is locally strongly convex (PL-type), implying that the gradient norm is proportional to the distance to the optimum and thus ensuring stable convergence.

$$\frac{1}{2} \|\nabla_W r_{\rho,n}(W)\|^2 \geq \mu(r_{\rho,n}(W) - r_{\rho,n}(W_n^*)) \quad (\forall W \in \mathcal{N}, \mu > 0). \quad (46)$$

(iii) **Slow schedule:** As shown in the previous section, the scheduling coefficient varies sufficiently slowly so that the optimization path remains stable (homotopy tracking).

(iv) **Reconstruction-dominant phase:** Lemma 3.1 stated in the main paper holds, and the early stage of training $[0, T_1]$ is dominated by the reconstruction loss.

PROOF.

Step 1: Per-step dominance carried to the handoff We analyze the handoff time t_1 when the objective transitions smoothly from the

surrogate loss to the NLL loss after the early training phase. By Assumption (iv), reconstruction remains dominant up to this time, so the magnitude of the reconstruction gradient g_{rec} is much larger than that of the constraint gradient g_h :

$$\|g_{\text{rec}}(t_1)\| = \lambda_{\text{reg}}(t_1) \|\nabla_W (\mathcal{L}_{\text{MSE}} + \mathcal{L}_{\text{VarReg}})\| \geq \frac{\kappa}{1-\delta} \|g_h(t_1)\|. \quad (47)$$

Here $\kappa \geq 1$ denotes the condition number of the Hessian $\nabla_W^2 \mathcal{L}_t$ (see Eq. (34)), and $\delta \in (0, 1)$ is the safety margin introduced in Lemma 3.1 stated in the main paper.

Step 2: Gradient decomposition and bound at t_1 Using the triangle inequality, we bound the full gradient $\nabla_W r_{\rho,n}(W_{t_1})$ by the sum of the component gradients. Combining this with Eq. (47), we derive an upper bound on the squared norm of the full gradient:

$$\|\nabla_W r_{\rho,n}(W_{t_1})\|^2 \leq C' \frac{\kappa \|g_h(t_1)\|^2}{\lambda_{\text{reg}}(t_1) \|\nabla_W (\mathcal{L}_{\text{MSE}} + \mathcal{L}_{\text{VarReg}})\|^2}, \quad (48)$$

for some constant $C' > 0$ that aggregates the smoothness constants of $r_{\rho,n}$ and the factor $(1-\delta)^{-2}$ introduced in Lemma 3.1 stated in the main paper. The key point is that the denominator's reconstruction signal is very large, so the overall gradient at the handoff is very small. This indicates that the optimization starts near a good solution.

Step 3: Empirical gap via PL condition Applying the PL condition in Assumption (ii), we show that the empirical error of the converged solution, $r_{\rho,n}(\hat{W}) - r_{\rho,n}(W_n^*)$, is proportional to the squared gradient norm. The small gradient magnitude from Step 2 directly yields a small empirical error:

$$\begin{aligned} r_{\rho,n}(\hat{W}) - r_{\rho,n}(W_n^*) &\leq \frac{1}{2\mu} \|\nabla_W r_{\rho,n}(\hat{W})\|^2 \\ &\leq C_2 \frac{\kappa \|\nabla_W \mathcal{L}_{\text{DAG}}(\hat{W})\|^2}{\lambda_{\text{reg}}(t_1) \|\nabla_W (\mathcal{L}_{\text{MSE}} + \mathcal{L}_{\text{VarReg}})(\hat{W})\|^2}. \end{aligned} \quad (49)$$

Here $C_2 > 0$ is a constant that depends only on C' from Eq. (48) and on the PL parameter μ in Eq. (46) (in particular, one may take $C_2 = C'/(2\mu)$).

Step 4: From empirical to population and the ξ -bound Finally, using the uniform convergence in Assumption (i), we transfer the small empirical error to a small population error. In particular, combining Eq. (45) with the empirical bound in Eq. (49) yields

$$r_\rho(\hat{W}) - r_\rho(W_\rho^*) \leq C_1 \Delta_n + C_2 \frac{\kappa \|\nabla_W \mathcal{L}_{\text{DAG}}(\hat{W})\|^2}{\lambda_{\text{reg}}(t_1) \|\nabla_W (\mathcal{L}_{\text{MSE}} + \mathcal{L}_{\text{VarReg}})(\hat{W})\|^2}, \quad (50)$$

for some numerical constant $C_1 \geq 1$ derived from the standard empirical-to-population decomposition; in our setting we can take $C_1 = 2$. This proves the conclusion of Claim 1 stated in the main paper that the error bound attainable by the proposed method (ξ_{ours}) is smaller than that of the standard NLL approach (ξ_{NLL}). $\square$

Remarks. In conclusion, the proposed optimization strategy stabilizes and amplifies the reconstruction signal in the early phase of training, ensuring that optimization starts from a favorable initialization. Consequently, it avoids constraint-induced early lock-in and enables the learning of more accurate DAG structures. The purpose of Claim 1 stated in the main paper, however, is not to provide a

fully general non-asymptotic guarantee that holds for arbitrary network architectures, optimization algorithms, or data distributions. Rather, it should be interpreted as an idealized, sample-dependent sufficient condition derived under the standard assumptions (i)–(iv) above—uniform convergence, a local PL-type condition along the data-induced solution path, a slow schedule for $\lambda_{\text{reg}}(t)$, and an early reconstruction-dominant phase. Within this setting, the bound in Eq. (16) of the main paper is meant to formalize the qualitative mechanism suggested by our experiments: by keeping the optimization trajectory in a reconstruction-dominant regime and ensuring a smooth transition to the NLL loss, the scheduling scheme can reduce the attainable error ξ_{ours} relative to ξ_{NLL} .

Regarding the assumptions themselves, Assumptions (iii) and (iv) are formalized using Assumption (i) and Lemma 3.1 (Eqs. (14)–(15)) stated in the main paper, and are further supported empirically by the analyses of the contribution ratio, the DAG constraint, and the optimization trajectory in Appendix C (Figure 1 of the main paper and Figures 7–11). These experiments show that our choice of schedule $\lambda_{\text{reg}}(t)$ indeed creates an early training phase in which the reconstruction loss dominates the DAG constraint, and that the transition from the surrogate loss to the NLL loss is sufficiently gradual in practice.

Assumptions (i) and (ii) follow the standard practice in the literature on homotopy and graduated non-convexity methods and PL-type convergence analyses [14, 18]. We use them as idealized conditions that allow us to relate empirical and population risks and to derive a formal sample-dependent bound. Verifying global uniform convergence and PL-type conditions for general nonlinear neural networks is intractable in full generality; instead, we adopt relaxed assumptions in the spirit of the above works and complement Claim stated in the main paper with empirical evidence for the behavior of our scheduling and optimization trajectory.

E Identifiability of Multivariate HNM

This section discusses the identifiability of HNM in the multivariate setting. As mentioned in Section 2 of the main paper, the identifiability condition of HNM in the multivariate setting follows previous studies [10, 48]. Therefore, the identifiability presented in this section is consistent with the identifiability theory established in prior work. We first analyze the identifiability of HNM in the bivariate setting and then extend it to the multivariate setting. HNM is identifiable if it satisfies the following conditions: (i) $\mu(\cdot)$ is a nonlinear function, (ii) $\sigma(\cdot)$ is a piecewise function, and (iii) the noise variables E are independent and Gaussian.

Assume that the variables X and Y follow the model in Eq (1),

$$Y = \mu_Y(X) + \sigma_Y(X)E_Y. \quad (51)$$

If a backward model exists, i.e.

$$X = \mu_X(Y) + \sigma_X(Y)E_X, \quad (52)$$

where E_X and E_Y are the independent exogenous noise variables following standard Gaussian distributions, μ_j and σ_j are twice-differentiable scalar functions with $\sigma_j(X), \sigma_j(Y) > 0$. For the forward and backward models to generate the same data distribution $p(X_1, X_2)$ simultaneously (i.e. for the model to be *unidentifiable*), one of the following scenarios must hold [22, 23]:

- (1) $(\sigma_Y, \mu_Y) = (\frac{1}{Q}, \frac{P}{Q})$ and $(\sigma_X, \mu_X) = (\frac{1}{Q'}, \frac{P'}{Q'})$ where Q, Q' are polynomials of degree two, $Q, Q' > 0$, P, P' are polynomials of degree two or less, and $p(Y), p(X)$ are strictly log-mix-rational-log.
- (2) $\sigma_j(X), \sigma_j(Y)$ are constant, μ_Y, μ_X are linear and $p(Y), p(X)$ are Gaussian densities.

In the HNM formulation we adopt in Eq. (1) of the main paper, μ_j is a nonlinear function, so the second scenario above (which requires linear μ) does not apply. Likewise, σ_j is piecewise (not of the form $\frac{1}{Q}$), so the first scenario does not apply. Therefore, there is no valid backward model for any distribution that satisfies Eq. (1) of the main paper for bivariate cases; in other words, the model is identifiable.

Building on *Proposition 28* and *Lemmas 35* and *36* of Peters et al. [33], we next examine identifiability in the multivariate setting. Assume that there exists another HNM with graph $\mathcal{G}'$ that $\mathcal{G} \neq \mathcal{G}'$. According to *Proposition 28* in Peters et al. [33]: let $\mathcal{G}$ and $\mathcal{G}'$ be two different DAGs over a set of variables X . Assume $p(X)$ is generated by HNM in Eq. (1) of the main paper and satisfies the Markov condition and causal minimality with respect to $\mathcal{G}$ and $\mathcal{G}'$. Then there are variables $L, Y \in X$ such that for the set $Q := \text{PA}_Y^{\mathcal{G}} \setminus \{L\}$, $R := \text{PA}_L^{\mathcal{G}'} \setminus \{L\}$ and $S := Q \cup R$, we have: (i) $L \rightarrow Y$ in $\mathcal{G}$ and $Y \rightarrow L$ in $\mathcal{G}'$. (ii) $S \subseteq \text{ND}_Y^{\mathcal{G}} \setminus \{L\}$ and $S \subseteq \text{ND}_L^{\mathcal{G}'} \setminus \{Y\}$. The set $\text{PA}_Y^{\mathcal{G}}$ consists of the parent variables of Y in $\mathcal{G}$ and $\text{ND}_Y^{\mathcal{G}}$ includes the non-descendants of Y in $\mathcal{G}'$.

Let $S = s$ for some value s such that $p(s) > 0$. Denote $L^* := L \mid S = s$ and $Y^* := Y \mid S = s$. According to Lemma 36 in [33], if the distribution $p(X)$ is generated by the following SEM:

$$X_j = f_j(X_{\text{pa}(j)}, E_j), \quad j = 1, 2, \dots, d, X_{\text{pa}(j)} \in X, \quad (53)$$

with corresponding DAG $\mathcal{G}$, then for any subset $K \subseteq \text{ND}_{X_j}^{\mathcal{G}}$, it holds that $E_{X_{\text{pa}(j)}} \perp\!\!\!\perp K$. Lemma 36 from Peters et al. [33] is applicable to HNM because an HNM can be viewed as one specific class of the SEM in Eq. (53) [48], and based on this, Yin et al. [48] showed the following holds: $E_Y \perp\!\!\!\perp (L, S)$ and $E_L \perp\!\!\!\perp (Y, S)$. We also follow these findings.

Lemma 35 from Peters et al. [33] indicates that if $E_Y \perp\!\!\!\perp (Y, Q, R)$ then this condition holds for all $\mathbf{q}, \mathbf{r}$ with $p(\mathbf{q}, \mathbf{r}) > 0$ and

$$f(Y, Q, E_Y) \mid_{Q=\mathbf{q}, R=\mathbf{r}} = f(Y \mid_{Q=\mathbf{q}, R=\mathbf{r}}, \mathbf{q}, E_Y). \quad (54)$$

Building on this, Yin et al. [48] derives the following expressions:

$$f(L, Q, E_Y) \mid_{S=s} = f(L \mid_{S=s}, E_Y) = f(L^*, \mathbf{q}, E_Y), \quad (55)$$

$$f(Y, R, E_L) \mid_{S=s} = f(Y \mid_{S=s}, E_L) = f(Y^*, \mathbf{r}, E_L). \quad (56)$$

From Eq. (1) of the main paper of the main paper, we then obtain the following result:

$$Y^* = \mu_Y(\mathbf{q}, L^*) + \sigma_Y(\mathbf{q}, L^*)E_Y, \quad E_Y \perp\!\!\!\perp L^* \text{ in } \mathcal{G}, \quad (57)$$

$$L^* = \mu_L(\mathbf{r}, Y^*) + \sigma_L(\mathbf{r}, Y^*)E_L, \quad E_L \perp\!\!\!\perp Y^* \text{ in } \mathcal{G}'. \quad (58)$$

Whereas no backward model satisfies the same assumptions as the forward model, Eqs. (57) and (58) holding simultaneously is inconsistent with this. Therefore, the assumption that there exists another HNM with $\mathcal{G} = \mathcal{G}'$ does not hold. As a result, the DAG $\mathcal{G}$ identifiable from $p(X)$ is unique.

Remark. Our method does not change the model class or the population objective itself. The surrogate loss together with its

Algorithm 1 GRADUATEDOPTIMIZATION: a weighted loss scheduling based L-BFGS optimization

Require: Input data X , parameters of the prediction network Θ , DAG parameter W , initial scheduling coefficient $\lambda_{\text{reg}}(0)$, transition point t^* , decay rate τ , current iteration t , Lagrange multiplier α , penalty coefficient ρ

Ensure: Learned parameters $\hat{\Theta}, \hat{W}$

```

1: repeat
2:   if  $t < t^*$  then                                 $\triangleright$  condition for transition
3:      $\lambda_{\text{reg}}(t) \leftarrow \lambda_{\text{reg}}(0) \cdot \exp\left(-\frac{t}{t^*/\tau}\right)$   $\triangleright$  scheduling (Eq. (8))
4:      $\mathcal{L}_{\text{surrogate}}(\Theta^{(t)}, W^{(t)}, t) \leftarrow \lambda_{\text{reg}}(t) \cdot$ 
        $((\mathcal{L}_{\text{MSE}}(\Theta_{\mu}^{(t)}, W^{(t)}) + \mathcal{L}_{\text{VarReg}}(\Theta^{(t)}, W^{(t)})) +$ 
        $\mathcal{L}_{\text{StopNLL}}(\Theta^{(t)}, W^{(t)}))$   $\triangleright$  surrogate loss (Eq. (7))
5:      $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{surrogate}}(\Theta^{(t)}, W^{(t)}, t) + \alpha \cdot h(W^{(t)}) +$ 
        $\frac{\rho}{2} h(W^{(t)})^2$   $\triangleright$  early-phase objective
6:   else
7:      $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{NLL}}(\Theta^{(t)}) + \alpha \cdot h(W^{(t)}) + \frac{\rho}{2} h(W^{(t)})^2$   $\triangleright$ 
       original objective (Eq. (3))
8:   end if
9:    $\Theta^{(t+1)}, W^{(t+1)} \leftarrow \text{L-BFGS}(\mathcal{L}_{\text{total}})$   $\triangleright$  update parameters via
       L-BFGS (with  $W_{ij} \geq 0$  and  $W_{ii} = 0$ ) [48, 55]
10:   $t \leftarrow t + 1$ 
11: until Convergence
12:  $\hat{\Theta} \leftarrow \Theta^{(t)}$ 
13:  $\hat{W} \leftarrow W^{(t)}$ 
14: return  $\hat{\Theta}, \hat{W}$ 

```

schedule serves as an auxiliary optimization mechanism introduced to improve the initial optimization trajectory: they guide the parameters toward a reconstruction-friendly regime and then, in the later stages of training, converge to the same Eq. (3) of the main paper as in Yin et al. [48]. The identifiability discussion under idealized conditions is built precisely on this standard NLL-based objective, and our method uses Eq. (3) of the main paper as is. Therefore, the proposed optimization strategy preserves the identifiability of HNMs at the population level, as it ultimately optimizes the same NLL-based objective. The difference lies in the optimization path, which is empirically more stable in finite-sample settings and leads to improved structure recovery compared to Yin et al. [48].

F Algorithm

Algorithm 1 illustrates our proposed graduated optimization method.

Algorithm 2 describes the overall training procedure based on augmented Lagrangian method (ALM). At the end of each L-BFGS inner loop, the penalty coefficients ρ and α are increased to enforce acyclicity on W , which is identical to the approach of Zheng et al. [55]. The training process terminates when ρ reaches ρ_{max} or $h(W)$ falls below the tolerance ϵ .

G Experiment Settings

For optimization, we used the NOTEARS [54] continuous optimization framework as the backbone, and all baseline methods were run with their default settings as given in the original papers. We adopted L-BFGS [4] for gradient-based training. All networks were

Algorithm 2 Augmented Lagrangian method-based DAG learning

Require: Initialized parameters for the prediction networks $\Theta^0 = \{\theta_{\mu}^0, \theta_{\sigma}^0\}$, Initialized DAG parameter W^0 , input data X , max outer iterations S , initial scheduling coefficient $\lambda_{\text{reg}}(0)$, transition point t^* , decay rate τ , tolerance ϵ , maximum penalty coefficient ρ_{max}

Ensure: Learned parameters Θ^*, W^*

```

1:  $h \leftarrow \infty, \alpha \leftarrow 0, \rho \leftarrow 1$   $\triangleright$  initialize Dual Ascent coefficients
2: global  $t \leftarrow 0$   $\triangleright$   $t$  counts inner-loop iterations
3: for  $s = 0$  to  $S$  do  $\triangleright$  start outer loop (outer iteration index  $s$ )
4:   while  $\rho < \rho_{\text{max}}$  do  $\triangleright$  inner loop: check penalty increase condition
5:      $\Theta^{(t)}, W^{(t)} \leftarrow \text{GRADUATEDOPTIMIZATION}(X, \Theta^{(t)}, W^{(t)},$ 
        $\lambda_{\text{reg}}(0), t^*, \tau, t, \alpha, \rho)$   $\triangleright$  inner loop optimization
6:      $h' \leftarrow h(W)$   $\triangleright$  compute DAG violations
7:     if  $h' > 0.25 \cdot h$  then
8:        $\rho \leftarrow \min(10 \cdot \rho, \rho_{\text{max}})$   $\triangleright$  increase penalty coefficient
9:     else
10:      break  $\triangleright$  stop inner loop
11:   end if
12:   end while
13:    $\alpha \leftarrow \alpha + \rho \cdot h'$   $\triangleright$  update Lagrange multiplier
14:    $h \leftarrow h'$ 
15:   if  $h \leq \epsilon$  or  $\rho \geq \rho_{\text{max}}$  then
16:     break  $\triangleright$  stop
17:   end if
18: end for
19:  $\Theta^* \leftarrow \Theta^{(t)}$ 
20:  $W^* \leftarrow W^{(t)}$ 
21: return  $\Theta^*, W^*$ 

```

2-layer MLPs with 10 hidden units, and the learned weight matrix W was binarized using the standard threshold of 0.3. In all experiments, we set $\tau = 5$, $\lambda_{\text{reg}}(0) = 100$, and $t^* = 1000$. We employed Structural Hamming distance (SHD), which measures edge differences between the estimated DAG and the ground-truth DAG, and F1-score of the edge detection, as our evaluation metrics.

We followed the settings of prior studies [24, 48, 49, 55] in generating synthetic data. The ground-truth causal graph was sampled from an Erdős-Rényi (ER) model with parameter $k \in \{1, 2\}$, which controls the average number of edges. We considered numbers of variables $d \in \{10, 20, 50, 100\}$ and generated 10 random graphs for each d . For each graph, we then generated $N = 1000$ observations, repeating this process 10 times.

G.1 Synthetic Dataset Description

To evaluate the performance of the proposed method, we generated nonlinear synthetic data with two types of noise (homoscedastic and heteroscedastic settings). The data generation process follows the standard setup used in Lachapelle et al. [24], Yin et al. [48], Zheng et al. [55]. Ground-truth DAGs were generated based on Erdős-Rényi (ER) random graphs [11]. Specifically, we randomly sampled a topological order of d variables and then added directed edges with probability $p = \frac{2k}{d^2 - d}$, where d is the number of variables and k is the expected number of edges in the DAG. We used ER

graphs with $k \in \{1, 2\}$, and for each setting of $d \in \{10, 20, 50, 100\}$, we generated 10 random graphs (using 10 different random seeds); each graph yielded $N = 1000$ observations.

Synthetic data with homoscedastic noise. Homoscedastic noise data is generated under the assumption that all variables have equal noise variance across all observations. In other words, each variable's noise has the same variance regardless of the observation. We generated such data by simulating a random DAG G and then, following its topological order, by drawing observations from the following nonlinear SEM:

$$X_j = \mu_j(X_{pa(j)}) + E_j, \quad j = 1, \dots, d, \quad (59)$$

where $\mu_j(\cdot)$ indicates a randomly initialized MLP with one hidden layer of size 100 and sigmoid activation. E_j are standard Gaussian noises, i.e. $E_j \sim \mathcal{N}(0, 1)$.

Synthetic data with heteroscedastic noise. Heteroscedastic noise data is generated under the assumption that noise variances differ across variables and across observations. In particular, we assume each variable's noise variance depends on its parent variables (as in Eq. (1) of the main paper). Using randomly generated DAGs G , we followed the topological order of G and drew observations from the following nonlinear SEM:

$$X_j = \mu_j(X_{pa(j)}) + \sigma_j(X_{pa(j)})E_j, \quad j = 1, \dots, d, \quad (60)$$

where $\sigma_j(\cdot) = \sqrt{\log(1 + \exp(g_j(\cdot)))}$, and $\mu_j(\cdot)$ and $g_j(\cdot)$ are randomly initialized MLPs with one hidden layer of size 100 and sigmoid activation. E_j are standard Gaussian noises, i.e. $E_j \sim \mathcal{N}(0, 1)$. Each variable's distribution in these simulations is Gaussian, and observations are sampled from these distributions. Because of this, inferring the exact structure from a single sample is challenging. Therefore, to perform effective inference when noise variances differ across the data, probabilistic modeling is essential, and using the NLL as the reconstruction loss is appropriate.

G.2 Baselines

We compared our proposed method with several representative DAG learning methods from prior work to evaluate its performance. The baselines were selected to cover methods that assume either homoscedastic or heteroscedastic noise and that support the multi-variate setting:

- (1) **NOTEARS-MLP** [55] is a continuous optimization-based method that learns a DAG, assuming nonlinear SEMs with homoscedastic noise.
- (2) **GOLEM** [29] is a continuous optimization-based method for linear Gaussian and non-Gaussian settings. We use GOLEM-EV for the synthetic homoscedastic experiments.
- (3) **HOST** [10] is a combinatorial optimization-based method that assumes HNM and first identifies a causal ordering based on each variable's conditional normality, then recovers the DAG via conditional independence tests.
- (4) **ICDH** [48] is a continuous optimization-based method that assumes HNM and introduces a two-phase iterative learning algorithm that separates the training of the mean and variance functions to overcome optimization difficulties.

G.3 Evaluation Metrics

To assess the accuracy of DAG learning, we use the following metrics. Let the ground-truth DAG be $\mathcal{G}^* = (\mathcal{V}, \mathcal{E}^*)$ and the estimated DAG be $\hat{\mathcal{G}} = (\mathcal{V}, \hat{\mathcal{E}})$, where $\mathcal{V} = \{1, \dots, d\}$ denotes the node set corresponding to the d variables of $X = [X_1, \dots, X_d]$. For a directed edge set E , define its skeleton (undirected edge set) by

$$\text{sk}(E) := \{\{i, j\} : (i \rightarrow j) \in E \text{ or } (j \rightarrow i) \in E\}.$$

Also define

$$\text{TP} = |\hat{\mathcal{E}} \cap \mathcal{E}^*|, \quad \text{FP} = |\hat{\mathcal{E}} \setminus \mathcal{E}^*|, \quad \text{FN} = |\mathcal{E}^* \setminus \hat{\mathcal{E}}|.$$

Here, TP (true positives) counts correctly predicted directed edges, FP (false positives) counts predicted directed edges not in the ground truth, and FN (false negatives) counts ground-truth directed edges missed by the predictor.

- **Structural Hamming Distance (SHD).** The number of edge additions, deletions, and reversals needed to transform $\hat{\mathcal{G}}$ into $\mathcal{G}^*$:

$$\text{SHD}(\hat{\mathcal{G}}, \mathcal{G}^*) = |\text{sk}(\hat{\mathcal{E}}) \triangle \text{sk}(\mathcal{E}^*)| + R,$$

where the reversal count

$$R = \left| \left\{ \{i, j\} \in \text{sk}(\hat{\mathcal{E}}) \cap \text{sk}(\mathcal{E}^*) : (i \rightarrow j) \in \hat{\mathcal{E}}, \right. \right. \\ \left. \left. (j \rightarrow i) \in \mathcal{E}^* \text{ or vice versa} \right\} \right|.$$

- **False discovery rate (FDR).** Proportion of false positives among predicted positives:

$$\text{FDR} = \frac{\text{FP}}{\text{TP} + \text{FP}} \quad (\text{defined as 0 if } \text{TP} + \text{FP} = 0).$$

- **True positive rate (TPR).** Proportion of true positives among actual positives:

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

- **F1-score.** Harmonic mean of precision and recall:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \\ \text{F1} = \frac{2 \text{TP}}{2 \text{TP} + \text{FP} + \text{FN}}.$$

G.4 Implementation Details

We implemented our DAG learning algorithm following the pseudocode given in Algorithms 1 and 2. Following prior work [48, 55], we set ϵ , the tolerance for the acyclicity constraint in ALM, to 10^{-8} ; the maximum penalty coefficient to $\rho_{\max} = 10^{16}$. We increased ρ by a factor of 10 at each update. We applied an ℓ_1 regularization on W (with coefficient 0.05) to encourage minimality of the learned graph, and an ℓ_2 regularization (with coefficient 0.005) to reduce model complexity. Following Zheng et al. [55], we extended the DAG parameter from $W \in \mathbb{R}^{d \times d}$ to $W \in \mathbb{R}^{d \times d \times m \times 2}$, where m is the dimension of a hidden representation Z (we set $m = 10$). This W is split into two parameters $W^+ \in \mathbb{R}^{d \times d \times m}$ and $W^- \in \mathbb{R}^{d \times d \times m}$, and we compute Z as:

$$Z = \text{sigmoid}(XW^+ - XW^-). \quad (61)$$

The continuous graph structure represented by W^+ and W^- is given by:

$$\text{matrix } \mathcal{G} = \|(W^+ - W^-)_{ij}\|_2 = \left(\sum_{r=1}^m \left(W_{ijr}^+ - W_{ijr}^- \right)^2 \right)^{1/2}. \quad (62)$$

Finally, we obtain a discrete adjacency matrix by thresholding $\mathcal{G}$; following Zheng et al. [55], we fix this threshold to 0.3. All experiments were conducted on a computer equipped with an Intel i9-11900K CPU, 128GB RAM, and an NVIDIA GeForce RTX 3090 GPU, with the operating system of Ubuntu 20.04.

G.5 More Details for Section 2.2 of the Main Paper

This section provides the detailed settings for the experiments shown in Section 2.2 of the main paper. The results reported in Section 2.2 of the main paper and Figure 1 of the main paper empirically demonstrate the phenomenon that can occur in DAG learning under an HNM assumption. We used an ER1 random graph setting with $d = 10$ and heteroscedastic noise, generating 10 different datasets (with 10 different random seeds). For a fair comparison, we fixed ρ and α for the duration of 1000 training steps, since otherwise each method would start increasing ρ and α at different times to enforce acyclicity, making it difficult to analyze the compared methods in the same environment. When computing SHD, motivated by Yin et al. [48], we did not use a single fixed threshold of 0.3; instead, we averaged the SHD over 100 threshold values evenly spaced between 0.1 and 0.75, in order to capture edges with very weak connections during the early training phase. To observe the general trend of each metric over training, we applied Gaussian smoothing to remove sharp spikes in the metric values; we note that this smoothing does not affect the overall results but makes the trends easier to interpret.

For the experimental results reported in Figure 1 of the main paper, we also used an ER1 graph with $d = 10$ under heteroscedastic noise. This experiment visualizes the trajectory of W and the loss landscape over training. Because $W \in \mathbb{R}^{d \times d}$, direct visualization was challenging; thus, we reduced the dimensionality of W via PCA. We added small perturbations (noise) to W in the reduced space, projected the perturbed parameters back to the original space, and evaluated the loss to plot landscape¹.

H Additional Experiments

H.1 Performance on Denser Graphs

Following prior work [10, 48], we have presented experiments on nonlinear synthetic data for ER1 and ER2, which correspond to relatively sparse graphs. However, in real-world scenarios the data’s causal structure can be represented by denser graphs, so it is necessary to evaluate DAG learning performance in such cases as well. This section evaluates the performance of our method on denser graphs (ER graphs with $k \in \{3, 4, 5\}$) under heteroscedastic noise.

¹Meanwhile, the resulting trajectory and loss landscape shown in the main paper would be difficult to interpret due to the dimensionality reduction. Therefore, in Figure 12, we projected the trajectories into the same two-dimensional space (obtained via PCA) to enable a direct comparison.

Tables 3 and 4 show the results for the SHD and F1-score metrics, respectively. Consistent with the experiments on the ER1 and ER2 data, the proposed method achieved the lowest SHD on these denser graphs compared to the baselines. Notably, in the ER5 setting, the proposed method obtained an SHD of 88.1, which is much lower than the value of 140.2 achieved by NOTEARS-MLP (the next best baseline). In terms of F1-score, the proposed method also consistently achieved the highest scores compared to the baselines, demonstrating that even in the denser graph setting it accurately identifies the true edges while effectively excluding spurious edges.

H.2 Performance on a Larger Graph (up to 1,000)

In the main text, we performed experiments on graphs with up to 100 nodes. To test whether performance remains stable even when the graph size becomes extremely large, we now evaluate our approach on a graph with 1,000 nodes. The results are shown in Table 5, 6. We observe that ICDH suffers from an explosive increase in SHD when the number of variables is very large. In contrast, our proposed method still performs much better than ICDH. Specifically, ICDH adds too many edges in an attempt to reconstruct the data, leading to a high SHD and a high FDR (false discovery rate), which in turn yields a low F1-score. NOTEARS-MLP, since it does not suffer from variance-related errors, produces fewer spurious edges than ICDH, but it likewise fails to properly reconstruct the data, resulting in a low F1 score. By contrast, our method is able to successfully reconstruct the data by placing a high weight on the reconstruction loss in the initial stage, achieving the highest F1-score and the lowest SHD among all methods. Of course, as the scale becomes this extreme, performance drops for all methods (including ours). However, even at this scale, our method maintains a significantly higher F1-score compared to the others.

H.3 Additional Result on Real-world Data

We evaluated the proposed method on another real-world dataset, Cause-Effect Pairs [35]. This dataset consists of 99 bivariate datasets collected from different domains, and for each pair the task is to infer the causal direction. The evaluation criterion is whether the inferred direction between two variables is correct ($X \leftarrow Y$ or $X \rightarrow Y$), and we use accuracy as the metric. To correct for potential bias caused by multiple pairs originating from the same source dataset [28], we also report a weighted accuracy. In computing the weighted accuracy, all pairs from the same source dataset are given equal weight such that the total weight for each dataset is 1.

Table 7 presents the experimental results. The proposed method correctly inferred 58 out of 99 causal directions. Most baselines inferred only 33 or 36 directions correctly; while the recently proposed ICDH method achieved 52, our method surpasses it. These results, together with those on the Sachs dataset, indicate that the proposed method achieves superior performance on real-world data compared to existing methods.

H.4 Further Analysis of Our Strategies

In this section, we further analyze the effect of the proposed surrogate loss and weighted loss scheduling. To this end, we compare the

Table 3: Comparison of SHD using denser graphs (ER3, ER4, ER5). Results are reported as mean $\pm$ standard deviation over 10 trials.

Methods	SHD ($\downarrow$)		
	ER3 $d = 20$	ER4 $d = 20$	ER5 $d = 30$
NOTEARS-MLP	57.4 $\pm$ 7.405	65.0 $\pm$ 5.899	140.2 $\pm$ 17.910
HOST	58.4 $\pm$ 3.442	77.5 $\pm$ 7.406	195.6 $\pm$ 17.568
ICDH	59.6 $\pm$ 17.414	74.6 $\pm$ 14.705	185.5 $\pm$ 44.592
Ours	30.6 $\pm$ 6.735	39.5 $\pm$ 5.937	88.1 $\pm$ 14.356

Table 4: Comparison of F1-score using denser graphs (ER3, ER4, ER5). Results are reported as mean $\pm$ standard deviation over 10 trials.

Methods	F1-score ($\uparrow$)		
	ER3 $d = 20$	ER4 $d = 20$	ER5 $d = 30$
NOTEARS-MLP	0.44 $\pm$ 0.074	0.45 $\pm$ 0.070	0.41 $\pm$ 0.046
HOST	0.40 $\pm$ 0.047	0.35 $\pm$ 0.068	0.27 $\pm$ 0.036
ICDH	0.58 $\pm$ 0.065	0.61 $\pm$ 0.062	0.55 $\pm$ 0.046
Ours	0.68 $\pm$ 0.101	0.68 $\pm$ 0.076	0.59 $\pm$ 0.093

Table 5: Comparison of SHD and F1-score using a larger graph (ER2). Results are reported as mean $\pm$ standard deviation over 10 trials.

Methods	SHD ($\downarrow$)	F1-score ($\uparrow$)
NOTEARS-MLP	1592.3 $\pm$ 219.8	0.18 $\pm$ 0.127
ICDH	5158.3 $\pm$ 598.1	0.09 $\pm$ 0.066
Ours	1130.5 $\pm$ 153.5	0.54 $\pm$ 0.093

Table 6: Comparison of FDR, TPR, and the number of edges using a larger graph (ER2). Results are reported as mean $\pm$ standard deviation over 10 trials.

Methods	FDR ($\downarrow$)	TPR ($\uparrow$)	# of edges
NOTEARS-MLP	0.89 $\pm$ 0.085	0.43 $\pm$ 0.142	2150 $\pm$ 237.61
ICDH	0.95 $\pm$ 0.033	0.64 $\pm$ 0.128	6404 $\pm$ 672.31
Ours	0.42 $\pm$ 0.087	0.51 $\pm$ 0.109	1552 $\pm$ 75.66

Table 7: Results on the Cause-Effect Pairs dataset.

Method	Accuracy $\uparrow$	Weighted Accuracy $\uparrow$
NOTEARS-MLP	39/99	0.49
NOTEARS	36/99	0.47
GOLEM-EV	33/99	0.40
GOLEM-NV	33/99	0.40
ICDH	52/99	0.58
Ours	58/99	0.66

accuracy of learned DAGs when using NLL [31], β -NLL [34], Faithful Heteroscedastic [41], and using only the surrogate loss. Then, we apply the weighted loss scheduling to each loss function and compare performance. This experiment is intended to analyze how

each component influences DAG learning performance and to evaluate the interaction between the surrogate loss and the weighted loss scheduling.

Table 8 shows the results in terms of SHD. First, comparing the loss functions without weighted loss scheduling, ‘Ours w/o $\dagger$ ’ (surrogate loss without transition and coefficient) achieved an SHD on par with β -NLL. This suggests that using the surrogate loss alone produces a similar effect on DAG learning as the loss functions proposed in the heteroscedastic regression literature [31, 34, 41]. Nevertheless, since its SHD is lower than that of NLL, such result implies that our surrogate loss is a more advantageous loss function for DAG learning than NLL. In other words, the surrogate loss can alleviate the difficulties arising from the joint optimization of NLL.

When applying the weighted loss scheduling strategy, the proposed method achieved the lowest SHD in all settings. For example, at $d = 50$, the proposed method attained an SHD of 22.6, whereas β -NLL obtained 28.0 and Faithful Heteroscedastic 55.5. Although all

Table 8: Additional analysis on our surrogate loss and weighted loss scheduling. Comparison of SHD for all baseline algorithms on ER1 (mean $\pm$ standard deviation over 10 trials). Here, 'w $\dagger$ ' indicates that weighted loss scheduling is applied.

Graph	Methods	SHD ($\downarrow$)		
		$d = 10$	$d = 20$	$d = 50$
ER1	NLL	8.6 ± 2.498	19.9 ± 6.220	47.7 ± 12.884
	β -NLL	5.0 ± 1.844	10.2 ± 2.676	30.8 ± 5.510
	Faithful	4.4 ± 2.289	10.2 ± 4.686	98.5 ± 40.038
	Ours w/o $\dagger$	4.5 ± 1.857	11.8 ± 4.556	32.5 ± 9.770
	NLL w $\dagger$	8.4 ± 3.323	16.8 ± 4.059	38.3 ± 5.352
	β -NLL w $\dagger$	4.0 ± 1.949	9.3 ± 3.743	28.0 ± 6.633
	Faithful w $\dagger$	4.3 ± 2.610	11.7 ± 4.900	55.5 ± 20.422
	Ours	3.8 ± 2.272	7.4 ± 3.262	22.6 ± 4.609

loss functions saw some performance improvement with weighted loss scheduling (denoted by 'w $\dagger$ '), applying this to the surrogate loss yielded an overwhelming performance gain. This suggests that the proposed surrogate loss is designed to effectively guide the optimization trajectory under the weighted loss scheduling strategy.

As a result, the surrogate loss is not simply aimed at stable likelihood optimization; beyond this, it is designed in anticipation of the training objective's gradual transition to the NLL with using weighted loss scheduling. This design is a distinguishing feature compared to the loss functions from the heteroscedastic regression literature, and ultimately it enables more accurate DAG learning.

H.5 Further Analysis of Weighted Loss Scheduling with the Constant Variance model

The central motivation of this paper is the optimization instability that occurs when using the NLL as the reconstruction loss — an issue that does not arise when using MSE. We empirically verified this phenomenon through the experiment in Section 2.2 of the main paper (Figure 1). In addition, we analyze the performance of applying our weighted loss scheduling to NOTEARS-MLP (which uses a constant variance). This entails multiplying the MSE term of NOTEARS-MLP by our weight factor that varies with each training step. We use the same hyperparameters for the scheduling, and the results are presented in Table 9. Applying the proposed weighted loss scheduling to NOTEARS-MLP consistently improved its performance. This indicates that training the model with a strongly emphasized reconstruction loss in the early stage can steer W closer to the true DAG, demonstrating how crucial the influence of the reconstruction loss is. Nonetheless, our proposed approach (using a surrogate loss with weighted loss scheduling) achieved even higher performance. This suggests that in an HNM setting a variance function is indeed necessary, and that one must not only predict the variance but also ensure that the reconstruction loss has a strong influence in the initial phase.

H.6 Comparison with Heterogeneous Data Methods

We discussed the differences between methods for handling heterogeneous data and HNM-based methods in Section A.4. Despite differences in the underlying assumptions, here we aim to clarify the distinctions by directly comparing their performance. We do not treat these results as our primary observations, and we do not interpret performance differences with these baselines as superiority, since they correspond to a different problem setting.

For this comparison, we selected DARING [19] and ReScore [51] as representative methods designed for heterogeneous data. We conducted experiments on synthetic data corresponding to ER1 and ER2 with $d = \{10, 20, 50, 100\}$ under the HNM setting. Table 10 shows the results in terms of SHD, and Table 11 shows the results in terms of F1-score. DARING estimates a DAG by assuming independence among residuals; however, under an HNM this assumption can be violated because the residual(noise) variance depends on the input, causing DARING's performance to deteriorate as the number of variables increases. Additionally, ReScore assigns weights based solely on the sample loss, which under HNM amplifies the score estimation bias caused by input-dependent variance, leading to degraded performance.

For the ER1 case with $d = 10$, the HNM-based methods HOST and ICDH show better performance than DARING and ReScore. This performance difference can be interpreted as a consequence of the different assumptions used by the methods. On the other hand, as the graph becomes more complex, the performance of HOST and ICDH worsens. This is exactly the limitation of those methods as defined in our paper. Because of the need to estimate variance information in HNMs, the model's reconstruction capability is unstable in the early stage. We analyzed this problem and proposed a methodology to address it. As a result, our method is able to maintain the best performance even as the graph structure becomes more complex.

H.7 Computational Cost

Time Complexity. From Zheng et al. [55], NOTEARS-MLP requires $O(Nd^2m + d^2m + d^3)$ floating-point operations per L-BFGS iteration, where N denotes the number of samples, d denotes the number of variables, and m denotes the hidden-layer width. When

Table 9: Comparison of SHD for NOTEARS-MLP (which does not predict variance), with and without the proposed weighted loss scheduling, on nonlinear synthetic heteroscedastic-noise datasets (mean $\pm$ standard deviation over 10 trials). $\dagger$ denotes weighted loss scheduling.

Graph	Methods	SHD ($\downarrow$)			
		$d = 10$	$d = 20$	$d = 50$	$d = 100$
ER1	NOTEARS-MLP	9.1 $\pm$ 4.323	19.1 $\pm$ 4.636	88.9 $\pm$ 23.851	275.6 $\pm$ 49.364
	NOTEARS-MLP w $\dagger$	5.1 $\pm$ 2.835	12.5 $\pm$ 3.923	64.5 $\pm$ 16.832	112.0 $\pm$ 19.955
	Ours	3.8 $\pm$ 2.272	7.4 $\pm$ 3.262	22.6 $\pm$ 4.609	46.7 $\pm$ 7.988
ER2	NOTEARS-MLP	12.8 $\pm$ 3.682	42.0 $\pm$ 8.136	150.8 $\pm$ 40.506	438.5 $\pm$ 77.740
	NOTEARS-MLP w $\dagger$	10.0 $\pm$ 3.184	32.3 $\pm$ 7.095	103.7 $\pm$ 29.947	308.5 $\pm$ 66.395
	Ours	7.0 $\pm$ 2.236	17.5 $\pm$ 5.408	48.5 $\pm$ 16.585	132.38 $\pm$ 50.838

Table 10: Comparison of SHD with heterogeneous-data methods in the heteroscedastic setting (mean $\pm$ standard deviation over 10 trials).

Graph	Methods	SHD ($\downarrow$)			
		$d = 10$	$d = 20$	$d = 50$	$d = 100$
ER1	DARING	7.3 $\pm$ 3.164	14.5 $\pm$ 4.433	59.6 $\pm$ 15.351	134.6 $\pm$ 17.500
	ReScore	7.4 $\pm$ 3.929	25.6 $\pm$ 10.707	311.7 $\pm$ 58.905	1044.1 $\pm$ 165.83
	HOST	5.4 $\pm$ 3.105	11.4 $\pm$ 2.458	84.4 $\pm$ 26.740	143.8 $\pm$ 21.085
	ICDH	6.7 $\pm$ 4.473	26.0 $\pm$ 11.645	303.9 $\pm$ 121.223	518.8 $\pm$ 85.588
	Ours	3.8 $\pm$ 2.272	7.4 $\pm$ 3.262	22.6 $\pm$ 4.609	46.7 $\pm$ 7.988
ER2	DARING	13.2 $\pm$ 3.970	37.0 $\pm$ 7.416	107.0 $\pm$ 17.748	271.8 $\pm$ 29.624
	ReScore	13.9 $\pm$ 3.330	57.1 $\pm$ 16.748	371.6 $\pm$ 104.348	1360.1 $\pm$ 298.263
	HOST	14.5 $\pm$ 3.138	38.8 $\pm$ 5.689	201.8 $\pm$ 63.871	495.7 $\pm$ 155.535
	ICDH	13.7 $\pm$ 4.001	50.0 $\pm$ 19.037	415.3 $\pm$ 185.616	664.0 $\pm$ 194.376
	Ours	7.0 $\pm$ 2.236	17.5 $\pm$ 5.408	48.5 $\pm$ 16.585	132.38 $\pm$ 50.838

focusing on the dependence with respect to d , the total time complexity simplifies to $O(d^3)$. ICDH is implemented under the same augmented Lagrangian framework and uses the same MLP backbone, so the per-iteration cost of evaluating its objective and gradient has the same order $O(Nd^2m + d^2m + d^3)$. However, ICDH employs a two-phase alternating optimization strategy in which the mean and variance networks are updated in separate phases. Let $S \geq 1$ denote the constant that summarizes the additional number of such two-phase alternations required to reach convergence compared to a single-phase update. Then the total complexity of ICDH can be expressed as $O(Sd^3)$.

Our method shares the same augmented Lagrangian backbone and 2-layer MLP architecture as NOTEARS-MLP and ICDH, so that per L-BFGS iteration incurs a cost of $O(Nd^2m + d^2m + d^3)$, and the additional surrogate loss terms involve only simple pointwise operations and a scalar update of $\lambda_{\text{reg}}(t)$, which do not change this asymptotic order. Unlike ICDH, however, our approach jointly updates the entire mean and variance networks in a single phase at every iteration, so there is no extra alternating factor. Consequently,

the overall complexity of our method over total optimization iterations is $O(d^3)$ with respect to d .

Empirical Analysis. To quantify the extra computational overhead incurred by the surrogate loss and the weighted loss scheduling, we compare our method against NOTEARS-MLP and ICDH in terms of wall-clock training time, total number of optimization iterations, and peak GPU memory usage on the ER2 synthetic graph with $d = 20$ under the HNM. All methods are implemented within the same augmented Lagrangian framework with L-BFGS updates and share identical two-layer MLP backbones and hardware configurations, so that the per-iteration computational complexity scales identically with the number of samples and variables. The surrogate loss and the scheduling mechanism only add simple pointwise computations ($\mathcal{L}_{\text{MSE}}$, $\mathcal{L}_{\text{VarReg}}$, $\mathcal{L}_{\text{StopNLL}}$) and a scalar update for $\lambda_{\text{reg}}(t)$, and thus do not change the asymptotic per-iteration complexity.

Table 12 reports the result. In terms of wall-clock time, our method takes 651.47 seconds, which is slower than NOTEARS-MLP (19.67 seconds). ICDH takes 389.85 seconds and is therefore faster than our method. However, while NOTEARS-MLP and our

Table 11: Comparison of F1-score with heterogeneous-data methods in the heteroscedastic setting (mean $\pm$ standard deviation over 10 trials).

Graph	Methods	F1-score ($\uparrow$)			
		$d = 10$	$d = 20$	$d = 50$	$d = 100$
ER1	DARING	0.58 ± 0.189	0.57 ± 0.145	0.46 ± 0.064	0.45 ± 0.057
	ReScore	0.66 ± 0.184	0.53 ± 0.088	0.20 ± 0.047	0.13 ± 0.023
	HOST	0.66 ± 0.193	0.67 ± 0.079	0.43 ± 0.084	0.46 ± 0.041
	ICDH	0.72 ± 0.171	0.54 ± 0.116	0.25 ± 0.133	0.11 ± 0.018
	Ours	0.79 ± 0.132	0.79 ± 0.111	0.74 ± 0.053	0.73 ± 0.052
ER2	DARING	0.54 ± 0.157	0.49 ± 0.088	0.42 ± 0.064	0.37 ± 0.033
	ReScore	0.65 ± 0.085	0.51 ± 0.099	0.29 ± 0.075	0.18 ± 0.031
	HOST	0.49 ± 0.103	0.46 ± 0.072	0.30 ± 0.064	0.27 ± 0.061
	ICDH	0.65 ± 0.085	0.55 ± 0.111	0.31 ± 0.135	0.17 ± 0.056
	Ours	0.79 ± 0.071	0.76 ± 0.087	0.71 ± 0.100	0.62 ± 0.080

Table 12: Computational cost on the ER2 synthetic graph with $d = 20$ under the heteroscedastic noise setting. We report wall-clock training time (in seconds), the total number of optimization iterations (in units of 10^3), and peak GPU memory usage (in MB) for each method.

Method	Time (s)	Iterations ($\times 10^3$)	Memory (MB)
NOTEARS-MLP	19.67	25.07	540.93
ICDH	389.85	131.88	542.87
NLL	1463.85	163.07	550.05
Ours	651.47	28.03	549.11

method require a comparable number of iterations, ICDH needs about 4.7 times more iterations, reaching 131.88M. This larger number of iterations for ICDH is consistent with our time-complexity analysis, according to which ICDH requires S times more iterations. Nevertheless, ICDH finishes training faster than our method because, in its two-phase training procedure, the gradients for the mean and variance are computed and updated separately, which is substantially cheaper than jointly updating all parameters. In contrast, our method updates the full mean and variance networks at every L-BFGS step and evaluates additional surrogate-loss terms, leading to a higher per-iteration cost but a more efficient trajectory in terms of the number of iterations required for convergence. We also measured the training time when we train from the beginning with the NLL objective; in that case, the training took much longer and required many more iterations than our method. This highlights the difficulty of modeling heteroscedastic noise while jointly learning the mean and variance networks. Because our method optimizes an easier objective than NLL and follows a smoother optimization trajectory, it appears to converge faster than directly optimizing the NLL.

H.8 Comprehensive Comparison with Variance estimation Methods

To further strengthen the empirical evaluation, we additionally compare with several DAG learning methods that include variance- or likelihood-based objectives: GraN-DAG [24], NPVAR [12], NOTEARS-ENT [5], DAGMA [2], and MCP [9]. These methods implicitly estimate variance or uncertainty through likelihood- or entropy-based losses and therefore use a negative log-likelihood reconstruction objective, which makes them natural baselines in our setting. However, unlike our method, they do not explicitly output an observation-wise variance via a separate network, nor are they derived under the HNM assumption—that is, they do not estimate variance as a function of the parent variables. Instead, variance (or scale) information is absorbed into their loss or regularization terms, and their optimization procedures are not designed to account for the interaction between variance estimation and the acyclicity constraint.

We evaluate all methods on nonlinear synthetic datasets in the HNM setting with ER1 and ER2 graphs and $d \in \{10, 20, 50, 100\}$, using the same two-layer MLP architecture and sigmoid activation, and keeping the default optimization settings of each method. Tables 13-16 report the results in terms of SHD, F1-score, TPR, and FDR.

As can be seen from Table 13, our method consistently achieves the lowest SHD across all graph types and dimensional settings. For ER1 graphs, the advantage of our method over the strongest variance-estimation baselines becomes more pronounced as the number of variables increases. For example, when $d = 100$, our method attains an SHD of 46.7, whereas the strongest baselines in this setting, DAGMA and NPVAR, achieve 81.4 and 66.1, respectively. A similar trend is observed for ER2 graphs: our method again yields the smallest SHD for all d , with particularly large gaps on larger graphs. In contrast, GraN-DAG attains very large SHD values,

indicating a tendency to produce severely misspecified structures under HNM.

These results show that merely using a DAG learner with a likelihood- or entropy-based objective that implicitly handles variance is not sufficient to guarantee robust structure learning under HNM. Although GraN-DAG, NPV AR, NOTEARS-ENT, DAGMA, and MCP exploit variance-related information through their objectives, they do not explicitly parameterize observation-wise variance and are not designed to account for the interaction between NLL and the acyclicity constraint in the early stage of training. As the graphs grow larger, these methods tend to overfit, leading to high FDR and SHD, or, conversely, to miss many true edges. In contrast, our method explicitly models the HNM and uses the proposed surrogate loss and graduated loss scheduling to stabilize the joint learning of the mean and variance in the early phase, and then slowly transitions to the standard NLL objective. This design contributes to consistently lower SHD and higher F1-scores than the variance-estimation baselines, especially in higher-dimensional and more complex graph settings.

I Supplementary of Figure 2 of the Main Paper

In Figure 2 of the main paper, we compared the performance of our method with the baselines on the synthetic datasets. This section provides more detailed results related to this.

I.1 Heteroscedastic Noise Datasets

Tables 17, 18, 19 and 20 present the experimental results on nonlinear heteroscedastic noise synthetic datasets in terms of the SHD, F1-score, TPR, and FDR metrics, respectively. The proposed method consistently achieved the best performance in SHD, F1-score, and FDR. Although ICDH attained the highest TPR, this is because ICDH produced more edges than the ground-truth DAG; this is evidenced by ICDH's extremely high FDR, indicating that it generates a graph with many spurious edges. In contrast, the proposed method achieved the lowest FDR, meaning it effectively filtered out spurious edges. Moreover, by achieving the highest F1-score, our method overall recovered a structure most similar to the true DAG.

I.2 Homoscedastic Noise Datasets

Tables 21, 22, 23 and 24 present the results in terms of the SHD, F1-score, TPR, and FDR metrics in the homoscedastic noise setting. The evaluation results show a similar trend in the homoscedastic noise environment. The proposed method achieved the best performance in terms of SHD, F1-score, and FDR, whereas ICDH recorded a high TPR but also an extremely high FDR, indicating that its prediction includes many spurious edges. In contrast, the proposed method recovered a structure that is overall the closest to the true DAG while excluding unnecessary edges.

J Supplementary Results for Table 1 in the Main Paper

In the main text Table 1, we performed experiments on real-world datasets in terms of the SHD and F1-score. In addition to this, Table 25 reports TPR and FDR. On the Sachs and SynTReN datasets, our method consistently achieves low FDR and high TPR, which is consistent with the SHD and F1-score results in Table 1 of the

main paper. On the Causal Assembly dataset, HOST attains the lowest FDR and highest TPR. This suggests that first learning a causal ordering of the variables and then reconstructing the DAG via conditional independence tests is effective in complex manufacturing systems. Among the methods that perform continuous DAG learning without separately learning an ordering, our method achieves the second-lowest FDR after HOST.

Table 13: Comparison of SHD results across variance estimation methods in the heteroscedastic setting (mean $\pm$ standard deviation over 10 trials).

Graph	Methods	SHD ($\downarrow$)			
		$d = 10$	$d = 20$	$d = 50$	$d = 100$
ER1	GraN-DAG	22.7 $\pm$ 8.179	91.6 $\pm$ 26.563	572.5 $\pm$ 74.911	2319.2 $\pm$ 316.701
	NPVAR	5.9 $\pm$ 3.635	14.2 $\pm$ 6.143	32.3 $\pm$ 3.592	66.1 $\pm$ 5.859
	NOTEARS-ENT	7.5 $\pm$ 3.240	17.5 $\pm$ 7.153	41.4 $\pm$ 6.415	103.5 $\pm$ 6.721
	DAGMA	5.0 $\pm$ 2.309	11.0 $\pm$ 4.619	31.7 $\pm$ 7.088	81.4 $\pm$ 19.478
	MCP	4.4 $\pm$ 2.591	12.3 $\pm$ 4.644	42.1 $\pm$ 10.671	139.9 $\pm$ 61.952
	Ours	3.8 $\pm$ 2.272	7.4 $\pm$ 3.262	22.6 $\pm$ 4.609	46.7 $\pm$ 7.988
ER2	GraN-DAG	20.4 $\pm$ 6.346	77.6 $\pm$ 20.866	438.3 $\pm$ 77.379	1459.5 $\pm$ 215.857
	NPVAR	14.2 $\pm$ 4.158	32.5 $\pm$ 5.642	80.8 $\pm$ 8.025	169.6 $\pm$ 8.444
	NOTEARS-ENT	12.0 $\pm$ 3.590	29.1 $\pm$ 7.909	83.9 $\pm$ 6.540	200.2 $\pm$ 14.266
	DAGMA	11.5 $\pm$ 2.799	22.1 $\pm$ 6.967	63.5 $\pm$ 9.301	146.9 $\pm$ 64.851
	MCP	9.8 $\pm$ 3.882	23.5 $\pm$ 5.855	72.1 $\pm$ 17.785	167.7 $\pm$ 23.386
	Ours	7.0 $\pm$ 2.236	17.5 $\pm$ 5.408	48.5 $\pm$ 16.585	132.38 $\pm$ 50.838

Table 14: Comparison of F1-score results across variance estimation methods in the heteroscedastic setting (mean $\pm$ standard deviation over 10 trials).

Graph	Methods	F1-score ($\uparrow$)			
		$d = 10$	$d = 20$	$d = 50$	$d = 100$
ER1	GraN-DAG	0.37 $\pm$ 0.133	0.23 $\pm$ 0.093	0.11 $\pm$ 0.023	0.06 $\pm$ 0.010
	NPVAR	0.62 $\pm$ 0.229	0.55 $\pm$ 0.181	0.57 $\pm$ 0.082	0.55 $\pm$ 0.050
	NOTEARS-ENT	0.65 $\pm$ 0.156	0.55 $\pm$ 0.146	0.48 $\pm$ 0.079	0.19 $\pm$ 0.052
	DAGMA	0.67 $\pm$ 0.192	0.66 $\pm$ 0.168	0.63 $\pm$ 0.080	0.60 $\pm$ 0.072
	MCP	0.73 $\pm$ 0.201	0.66 $\pm$ 0.136	0.59 $\pm$ 0.057	0.51 $\pm$ 0.094
	Ours	0.79 $\pm$ 0.132	0.79 $\pm$ 0.111	0.74 $\pm$ 0.053	0.73 $\pm$ 0.052
ER2	GraN-DAG	0.54 $\pm$ 0.115	0.38 $\pm$ 0.067	0.24 $\pm$ 0.046	0.16 $\pm$ 0.022
	NPVAR	0.47 $\pm$ 0.171	0.48 $\pm$ 0.105	0.45 $\pm$ 0.071	0.41 $\pm$ 0.029
	NOTEARS-ENT	0.65 $\pm$ 0.111	0.59 $\pm$ 0.132	0.40 $\pm$ 0.073	0.18 $\pm$ 0.068
	DAGMA	0.59 $\pm$ 0.133	0.64 $\pm$ 0.164	0.58 $\pm$ 0.075	0.60 $\pm$ 0.043
	MCP	0.68 $\pm$ 0.143	0.67 $\pm$ 0.111	0.59 $\pm$ 0.070	0.56 $\pm$ 0.036
	Ours	0.79 $\pm$ 0.071	0.76 $\pm$ 0.087	0.71 $\pm$ 0.100	0.62 $\pm$ 0.080

Table 15: Comparison of TPR results across variance estimation methods in the heteroscedastic setting (mean $\pm$ standard deviation over 10 trials).

Graph	Methods	TPR ($\uparrow$)			
		$d = 10$	$d = 20$	$d = 50$	$d = 100$
ER1	GraN-DAG	0.70 $\pm$ 0.200	0.67 $\pm$ 0.103	0.69 $\pm$ 0.076	0.69 $\pm$ 0.072
	NPVAR	0.56 $\pm$ 0.196	0.51 $\pm$ 0.158	0.53 $\pm$ 0.102	0.49 $\pm$ 0.046
	NOTEARS-ENT	0.76 $\pm$ 0.196	0.59 $\pm$ 0.160	0.41 $\pm$ 0.086	0.13 $\pm$ 0.041
	DAGMA	0.59 $\pm$ 0.223	0.59 $\pm$ 0.205	0.57 $\pm$ 0.105	0.62 $\pm$ 0.091
	MCP	0.69 $\pm$ 0.233	0.68 $\pm$ 0.190	0.64 $\pm$ 0.099	0.69 $\pm$ 0.088
	Ours	0.77 $\pm$ 0.127	0.76 $\pm$ 0.157	0.71 $\pm$ 0.069	0.66 $\pm$ 0.073
ER2	GraN-DAG	0.69 $\pm$ 0.116	0.63 $\pm$ 0.071	0.69 $\pm$ 0.078	0.69 $\pm$ 0.031
	NPVAR	0.41 $\pm$ 0.178	0.46 $\pm$ 0.134	0.40 $\pm$ 0.071	0.35 $\pm$ 0.028
	NOTEARS-ENT	0.67 $\pm$ 0.092	0.59 $\pm$ 0.159	0.30 $\pm$ 0.071	0.11 $\pm$ 0.047
	DAGMA	0.45 $\pm$ 0.127	0.54 $\pm$ 0.188	0.48 $\pm$ 0.097	0.53 $\pm$ 0.053
	MCP	0.59 $\pm$ 0.133	0.66 $\pm$ 0.139	0.57 $\pm$ 0.094	0.57 $\pm$ 0.054
	Ours	0.71 $\pm$ 0.092	0.72 $\pm$ 0.109	0.63 $\pm$ 0.092	0.54 $\pm$ 0.033

Table 16: Comparison of FDR results across variance estimation methods in the heteroscedastic setting (mean $\pm$ standard deviation over 10 trials).

Graph	Methods	FDR ($\downarrow$)			
		$d = 10$	$d = 20$	$d = 50$	$d = 100$
ER1	GraN-DAG	0.74 $\pm$ 0.106	0.86 $\pm$ 0.070	0.94 $\pm$ 0.013	0.97 $\pm$ 0.005
	NPVAR	0.29 $\pm$ 0.308	0.38 $\pm$ 0.234	0.37 $\pm$ 0.071	0.35 $\pm$ 0.071
	NOTEARS-ENT	0.43 $\pm$ 0.143	0.47 $\pm$ 0.149	0.43 $\pm$ 0.094	0.63 $\pm$ 0.071
	DAGMA	0.17 $\pm$ 0.124	0.20 $\pm$ 0.173	0.29 $\pm$ 0.117	0.42 $\pm$ 0.083
	MCP	0.20 $\pm$ 0.140	0.31 $\pm$ 0.178	0.44 $\pm$ 0.098	0.58 $\pm$ 0.132
	Ours	0.18 $\pm$ 0.154	0.16 $\pm$ 0.092	0.22 $\pm$ 0.043	0.18 $\pm$ 0.070
ER2	GraN-DAG	0.54 $\pm$ 0.128	0.72 $\pm$ 0.064	0.85 $\pm$ 0.030	0.91 $\pm$ 0.014
	NPVAR	0.42 $\pm$ 0.204	0.48 $\pm$ 0.117	0.48 $\pm$ 0.075	0.50 $\pm$ 0.039
	NOTEARS-ENT	0.36 $\pm$ 0.143	0.41 $\pm$ 0.124	0.39 $\pm$ 0.072	0.58 $\pm$ 0.132
	DAGMA	0.12 $\pm$ 0.147	0.20 $\pm$ 0.117	0.25 $\pm$ 0.079	0.27 $\pm$ 0.062
	MCP	0.20 $\pm$ 0.183	0.31 $\pm$ 0.116	0.37 $\pm$ 0.088	0.43 $\pm$ 0.072
	Ours	0.12 $\pm$ 0.080	0.20 $\pm$ 0.070	0.19 $\pm$ 0.125	0.29 $\pm$ 0.190

Table 17: SHD results in the heteroscedastic setting (mean $\pm$ standard deviation over 10 trials).

Graph	Methods	SHD ($\downarrow$)			
		$d = 10$	$d = 20$	$d = 50$	$d = 100$
ER1	NOTEARS-MLP	9.1 ± 4.323	19.1 ± 4.636	88.9 ± 23.851	275.6 ± 49.364
	HOST	5.4 ± 3.105	11.4 ± 2.458	84.4 ± 26.740	143.8 ± 21.085
	ICDH	6.7 ± 4.473	26.0 ± 11.645	303.9 ± 121.223	518.8 ± 85.588
	SkewScore	7.9 ± 4.040	24.9 ± 7.724	133.8 ± 14.498	465.4 ± 29.033
	Ours	3.8 ± 2.272	7.4 ± 3.262	22.6 ± 4.609	46.7 ± 7.988
ER2	NOTEARS-MLP	12.8 ± 3.682	42.0 ± 8.136	150.8 ± 40.506	438.5 ± 77.740
	HOST	14.5 ± 3.138	38.8 ± 5.689	201.8 ± 63.871	495.7 ± 155.535
	ICDH	13.7 ± 4.001	50.0 ± 19.037	415.3 ± 185.616	664.0 ± 194.376
	SkewScore	15.2 ± 3.225	50.2 ± 12.164	201.5 ± 22.746	608.5 ± 43.224
	Ours	7.0 ± 2.236	17.5 ± 5.408	48.5 ± 16.585	132.38 ± 50.838

Table 18: F1-score results in the heteroscedastic setting (mean $\pm$ standard deviation over 10 trials).

Graph	Methods	F1-score ($\uparrow$)			
		$d = 10$	$d = 20$	$d = 50$	$d = 100$
ER1	NOTEARS-MLP	0.54 ± 0.211	0.52 ± 0.125	0.38 ± 0.070	0.30 ± 0.044
	HOST	0.66 ± 0.193	0.67 ± 0.079	0.43 ± 0.084	0.46 ± 0.041
	ICDH	0.72 ± 0.171	0.54 ± 0.116	0.25 ± 0.133	0.11 ± 0.018
	SkewScore	0.57 ± 0.190	0.48 ± 0.138	0.32 ± 0.023	0.21 ± 0.025
	Ours	0.79 ± 0.132	0.79 ± 0.111	0.74 ± 0.053	0.73 ± 0.052
ER2	NOTEARS-MLP	0.57 ± 0.143	0.47 ± 0.083	0.36 ± 0.068	0.29 ± 0.038
	HOST	0.49 ± 0.103	0.46 ± 0.072	0.30 ± 0.064	0.27 ± 0.061
	ICDH	0.65 ± 0.085	0.55 ± 0.111	0.31 ± 0.135	0.17 ± 0.056
	SkewScore	0.52 ± 0.101	0.43 ± 0.078	0.31 ± 0.049	0.24 ± 0.027
	Ours	0.79 ± 0.071	0.76 ± 0.087	0.71 ± 0.100	0.62 ± 0.080

Table 19: TPR results in the heteroscedastic setting (mean $\pm$ standard deviation over 10 trials).

Graph	Methods	TPR ($\uparrow$)			
		$d = 10$	$d = 20$	$d = 50$	$d = 100$
ER1	NOTEARS-MLP	0.57 ± 0.219	0.58 ± 0.153	0.57 ± 0.059	0.61 ± 0.079
	HOST	0.60 ± 0.190	0.66 ± 0.092	0.65 ± 0.081	0.66 ± 0.065
	ICDH	0.83 ± 0.149	0.78 ± 0.174	0.81 ± 0.068	0.82 ± 0.045
	SkewScore	0.63 ± 0.195	0.63 ± 0.142	0.65 ± 0.045	0.65 ± 0.053
	Ours	0.77 ± 0.127	0.76 ± 0.157	0.71 ± 0.069	0.66 ± 0.073
ER2	NOTEARS-MLP	0.50 ± 0.151	0.51 ± 0.066	0.44 ± 0.066	0.45 ± 0.020
	HOST	0.43 ± 0.112	0.48 ± 0.069	0.46 ± 0.056	0.47 ± 0.042
	ICDH	0.70 ± 0.100	0.78 ± 0.148	0.78 ± 0.067	0.77 ± 0.087
	SkewScore	0.51 ± 0.131	0.55 ± 0.104	0.51 ± 0.070	0.51 ± 0.043
	Ours	0.71 ± 0.092	0.72 ± 0.109	0.63 ± 0.092	0.54 ± 0.033

Table 20: FDR results in the heteroscedastic setting (mean $\pm$ standard deviation over 10 trials).

Graph	Methods	FDR ($\downarrow$)			
		$d = 10$	$d = 20$	$d = 50$	$d = 100$
ER1	NOTEARS-MLP	0.47 ± 0.248	0.54 ± 0.112	0.71 ± 0.070	0.80 ± 0.035
	HOST	0.27 ± 0.210	0.31 ± 0.077	0.67 ± 0.100	0.65 ± 0.041
	ICDH	0.35 ± 0.201	0.56 ± 0.122	0.84 ± 0.111	0.94 ± 0.010
	SkewScore	0.46 ± 0.199	0.60 ± 0.143	0.79 ± 0.018	0.87 ± 0.016
	Ours	0.18 ± 0.154	0.16 ± 0.092	0.22 ± 0.043	0.18 ± 0.070
ER2	NOTEARS-MLP	0.33 ± 0.156	0.56 ± 0.106	0.69 ± 0.081	0.79 ± 0.037
	HOST	0.42 ± 0.141	0.55 ± 0.089	0.77 ± 0.061	0.81 ± 0.055
	ICDH	0.39 ± 0.108	0.55 ± 0.154	0.79 ± 0.136	0.90 ± 0.048
	SkewScore	0.46 ± 0.143	0.64 ± 0.082	0.77 ± 0.040	0.84 ± 0.019
	Ours	0.12 ± 0.080	0.20 ± 0.070	0.19 ± 0.125	0.29 ± 0.190

Table 21: SHD results in the homoscedastic setting (mean $\pm$ standard deviation over 10 trials).

Graph	Methods	SHD ($\downarrow$)			
		$d = 10$	$d = 20$	$d = 50$	$d = 100$
ER1	NOTEARS-MLP	4.9 ± 2.071	12.6 ± 2.764	33.2 ± 5.758	62.0 ± 9.726
	GOLEM-EV	8.0 ± 1.732	25.8 ± 6.508	78.1 ± 8.994	159.5 ± 7.487
	HOST	4.1 ± 1.221	14.8 ± 7.373	62.8 ± 19.302	132.9 ± 19.429
	ICDH	3.8 ± 1.778	7.3 ± 4.051	23.7 ± 7.403	111.5 ± 36.456
	SkewScore	3.9 ± 1.322	15.9 ± 8.421	72.3 ± 23.211	156.5 ± 32.559
	Ours	3.3 ± 1.792	6.7 ± 3.716	15.7 ± 4.196	43.6 ± 8.273
ER2	NOTEARS-MLP	12.6 ± 3.800	29.4 ± 5.024	72.7 ± 10.696	151.9 ± 17.524
	GOLEM-EV	18.0 ± 2.049	42.8 ± 5.154	117.0 ± 6.856	239.2 ± 10.971
	HOST	13.8 ± 4.142	43.4 ± 10.200	244.6 ± 58.563	462.8 ± 81.906
	ICDH	6.0 ± 2.608	17.4 ± 4.652	79.3 ± 16.273	332.7 ± 83.568
	SkewScore	14.6 ± 4.621	45.1 ± 11.352	226.7 ± 60.201	482.6 ± 85.841
	Ours	5.7 ± 2.795	14.5 ± 6.136	33.7 ± 6.709	88.0 ± 10.392

Table 22: F1-score results in the homoscedastic setting (mean $\pm$ standard deviation over 10 trials).

Graph	Methods	F1-score ($\uparrow$)			
		$d = 10$	$d = 20$	$d = 50$	$d = 100$
ER1	NOTEARS-MLP	0.68 ± 0.161	0.60 ± 0.094	0.63 ± 0.034	0.65 ± 0.056
	GOLEM-EV	0.42 ± 0.156	0.25 ± 0.108	0.27 ± 0.066	0.26 ± 0.045
	HOST	0.74 ± 0.087	0.59 ± 0.138	0.51 ± 0.078	0.48 ± 0.037
	ICDH	0.80 ± 0.104	0.81 ± 0.113	0.78 ± 0.051	0.61 ± 0.075
	SkewScore	0.72 ± 0.095	0.54 ± 0.125	0.51 ± 0.081	0.45 ± 0.069
	Ours	0.81 ± 0.111	0.80 ± 0.121	0.83 ± 0.040	0.74 ± 0.061
ER2	NOTEARS-MLP	0.60 ± 0.114	0.53 ± 0.086	0.55 ± 0.054	0.55 ± 0.045
	GOLEM-EV	0.32 ± 0.096	0.24 ± 0.080	0.24 ± 0.054	0.24 ± 0.020
	HOST	0.54 ± 0.119	0.42 ± 0.098	0.28 ± 0.071	0.29 ± 0.042
	ICDH	0.85 ± 0.070	0.79 ± 0.066	0.69 ± 0.037	0.52 ± 0.067
	SkewScore	0.57 ± 0.139	0.39 ± 0.115	0.26 ± 0.065	0.25 ± 0.051
	Ours	0.83 ± 0.124	0.79 ± 0.094	0.81 ± 0.043	0.74 ± 0.037

Table 23: TPR results in the homoscedastic setting (mean $\pm$ standard deviation over 10 trials).

Graph	Methods	TPR ($\uparrow$)			
		$d = 10$	$d = 20$	$d = 50$	$d = 100$
ER1	NOTEARS-MLP	0.59 ± 0.170	0.52 ± 0.123	0.61 ± 0.059	0.62 ± 0.062
	GOLEM-EV	0.32 ± 0.147	0.22 ± 0.092	0.30 ± 0.071	0.29 ± 0.057
	HOST	0.68 ± 0.117	0.58 ± 0.119	0.69 ± 0.055	0.66 ± 0.050
	ICDH	0.83 ± 0.090	0.83 ± 0.119	0.88 ± 0.062	0.85 ± 0.037
	SkewScore	0.71 ± 0.126	0.63 ± 0.135	0.70 ± 0.054	0.69 ± 0.042
	Ours	0.77 ± 0.135	0.76 ± 0.132	0.82 ± 0.074	0.70 ± 0.064
ER2	NOTEARS-MLP	0.51 ± 0.104	0.44 ± 0.092	0.48 ± 0.051	0.49 ± 0.040
	GOLEM-EV	0.23 ± 0.071	0.18 ± 0.064	0.20 ± 0.046	0.19 ± 0.015
	HOST	0.48 ± 0.117	0.45 ± 0.085	0.49 ± 0.068	0.49 ± 0.048
	ICDH	0.90 ± 0.114	0.84 ± 0.102	0.90 ± 0.026	0.88 ± 0.032
	SkewScore	0.49 ± 0.110	0.49 ± 0.086	0.52 ± 0.072	0.48 ± 0.045
	Ours	0.78 ± 0.163	0.72 ± 0.115	0.76 ± 0.061	0.66 ± 0.039

Table 24: FDR results in the homoscedastic setting (mean $\pm$ standard deviation over 10 trials).

Graph	Methods	FDR ($\downarrow$)			
		$d = 10$	$d = 20$	$d = 50$	$d = 100$
ER1	NOTEARS-MLP	0.16 ± 0.160	0.26 ± 0.095	0.33 ± 0.083	0.31 ± 0.069
	GOLEM-EV	0.36 ± 0.134	0.67 ± 0.185	0.75 ± 0.064	0.76 ± 0.037
	HOST	0.16 ± 0.119	0.38 ± 0.173	0.59 ± 0.094	0.62 ± 0.036
	ICDH	0.23 ± 0.127	0.21 ± 0.124	0.28 ± 0.097	0.52 ± 0.084
	SkewScore	0.27 ± 0.237	0.53 ± 0.206	0.60 ± 0.102	0.67 ± 0.077
	Ours	0.14 ± 0.101	0.14 ± 0.117	0.16 ± 0.045	0.21 ± 0.069
ER2	NOTEARS-MLP	0.21 ± 0.219	0.30 ± 0.125	0.34 ± 0.088	0.37 ± 0.065
	GOLEM-EV	0.40 ± 0.155	0.59 ± 0.120	0.68 ± 0.065	0.68 ± 0.038
	HOST	0.38 ± 0.146	0.59 ± 0.123	0.80 ± 0.066	0.80 ± 0.036
	ICDH	0.18 ± 0.071	0.25 ± 0.057	0.43 ± 0.053	0.63 ± 0.072
	SkewScore	0.32 ± 0.450	0.68 ± 0.163	0.83 ± 0.058	0.83 ± 0.047
	Ours	0.12 ± 0.076	0.12 ± 0.077	0.13 ± 0.030	0.16 ± 0.036

Table 25: FDR and TPR results on the real-world datasets.

Method	Sachs ($d = 11$)		SynTReN ($d = 20$)		CausalAssembly ($d = 40$)	
	FDR($\downarrow$)	TPR($\uparrow$)	FDR($\downarrow$)	TPR($\uparrow$)	FDR($\downarrow$)	TPR($\uparrow$)
NOTEARS-MLP	0.72 ± 0.027	0.25 ± 0.017	0.92 ± 0.043	0.37 ± 0.191	0.92 ± 0.049	0.06 ± 0.046
HOST	0.68 ± 0.024	0.35 ± 0.015	0.91 ± 0.043	0.39 ± 0.176	0.76 ± 0.028	0.13 ± 0.042
ICDH	0.71 ± 0.028	0.38 ± 0.018	0.92 ± 0.045	0.38 ± 0.166	0.91 ± 0.029	0.13 ± 0.050
Ours	0.55 ± 0.031	0.41 ± 0.015	0.88 ± 0.062	0.40 ± 0.209	0.87 ± 0.065	0.11 ± 0.036

References

- [1] Eugene L Allgower and Kurt Georg. 2003. *Introduction to numerical continuation methods*. SIAM.
- [2] Kevin Bello, Bryon Aragam, and Pradeep Ravikumar. 2022. Dagma: Learning dags via m-matrices and a log-determinant acyclicity characterization. *Advances in Neural Information Processing Systems* 35 (2022), 8226–8239.
- [3] Patrick Blöbaum, Dominik Janzing, Takashi Washio, Shohei Shimizu, and Bernhard Schölkopf. 2018. Cause-effect inference by comparing regression errors. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 900–909.
- [4] Richard H Byrd, Peihuang Lu, Jorge Nocedal, and Ciyu Zhu. 1995. A limited memory algorithm for bound constrained optimization. *SIAM Journal on scientific computing* 16, 5 (1995), 1190–1208.
- [5] Weilin Chen, Jie Qiao, Ruichu Cai, and Zhifeng Hao. 2023. On the role of entropy-based loss for learning causal structure with continuous optimization. *IEEE Transactions on Neural Networks and Learning Systems* (2023).
- [6] David Maxwell Chickering. 2002. Optimal structure identification with greedy search. *Journal of machine learning research* 3, Nov (2002), 507–554.
- [7] Diego Colombo, Marloes H Maathuis, Markus Kalisch, and Thomas S Richardson. 2012. Learning high-dimensional directed acyclic graphs with latent and selection variables. *The Annals of Statistics* (2012), 294–321.
- [8] Chang Deng, Kevin Bello, Pradeep Ravikumar, and Bryon Aragam. 2023. Global optimality in bivariate gradient-based DAG learning. *Advances in Neural Information Processing Systems* 36 (2023), 17929–17968.
- [9] Chang Deng, Kevin Bello, Pradeep Ravikumar, and Bryon Aragam. 2024. Markov equivalence and consistency in differentiable structure learning. *Advances in Neural Information Processing Systems* 37 (2024), 91756–91797.
- [10] Bao Duong and Thin Nguyen. 2023. Heteroscedastic causal structure learning. In *ECAI 2023*. IOS Press, 598–605.
- [11] P ERDdS and A R&w. 1959. On random graphs I. *Publ. math. debrecen* 6, 290–297 (1959), 18.
- [12] Ming Gao, Yi Ding, and Bryon Aragam. 2020. A polynomial-time algorithm for learning nonparametric causal graphs. *Advances in Neural Information Processing Systems* 33 (2020), 11599–11611.
- [13] Yinghua Gao, Li Shen, and Shu-Tao Xia. 2021. Dag-gan: Causal structure learning with generative adversarial nets. In *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 3320–3324.
- [14] Matilde Gargiani, Andrea Zanelli, Quoc Tran-Dinh, Moritz Diehl, and Frank Hutter. 2020. Convergence analysis of homotopy-sgd for non-convex optimization. *arXiv preprint arXiv:2011.10298* (2020).
- [15] AmirEmad Ghassami, Negar Kiyavash, Biwei Huang, and Kun Zhang. 2018. Multi-domain causal structure learning in linear systems. *Advances in neural information processing systems* 31 (2018).
- [16] Clark Glymour, Kun Zhang, and Peter Spirtes. 2019. Review of causal discovery methods based on graphical models. *Frontiers in genetics* 10 (2019), 524.
- [17] Ruocheng Guo, Lu Cheng, Jundong Li, P Richard Hahn, and Huan Liu. 2020. A survey of learning causality with data: Problems and methods. *ACM Computing Surveys (CSUR)* 53, 4 (2020), 1–37.
- [18] Elad Hazan, Kfir Yehuda Levy, and Shai Shalev-Shwartz. 2016. On graduated optimization for stochastic non-convex problems. In *International conference on machine learning*. PMLR, 1833–1841.
- [19] Yue He, Peng Cui, Zheyang Shen, Renzhe Xu, Furui Liu, and Yong Jiang. 2021. Daring: Differentiable causal discovery with residual independence. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*. 596–605.
- [20] Biwei Huang, Kun Zhang, Jiji Zhang, Joseph Ramsey, Ruben Sanchez-Romero, Clark Glymour, and Bernhard Schölkopf. 2020. Causal discovery from heterogeneous/nonstationary data. *Journal of Machine Learning Research* 21, 89 (2020), 1–53.
- [21] Yuma Ichikawa. 2024. Controlling continuous relaxation for combinatorial optimization. *Advances in Neural Information Processing Systems* 37 (2024), 47189–47216.
- [22] Alexander Immer, Christoph Schultheiss, Julia E Vogt, Bernhard Schölkopf, Peter Bühlmann, and Alexander Marx. 2023. On the identifiability and estimation of causal location-scale noise models. In *International Conference on Machine Learning*. PMLR, 14316–14332.
- [23] Ilyes Khemakhem, Ricardo Monti, Robert Leech, and Aapo Hyvärinen. 2021. Causal autoregressive flows. In *International conference on artificial intelligence and statistics*. PMLR, 3520–3528.
- [24] Sébastien Lachapelle, Philippe Brouillard, Tristan Deleu, and Simon Lacoste-Julien. 2020. Gradient-Based Neural DAG Learning. In *International Conference on Learning Representations*.
- [25] Huu Le and Christopher Zach. 2020. A graduated filter method for large scale robust estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 5559–5568.
- [26] Loka Li, Ignavier Ng, Gongxu Luo, Biwei Huang, Guangyi Chen, Tongliang Liu, Bin Gu, and Kun Zhang. 2024. Federated causal discovery from heterogeneous data. *arXiv preprint arXiv:2402.13241* (2024).
- [27] Yingyu Lin, Yuxing Huang, Wenqin Liu, Haoran Deng, Ignavier Ng, Kun Zhang, Mingming Gong, Yian Ma, and Biwei Huang. 2025. A skewness-based criterion for addressing heteroscedastic noise in causal discovery. In *International Conference on Learning Representations*, Vol. 2025. 89283–89310.
- [28] Joris M Mooij, Jonas Peters, Dominik Janzing, Jakob Zscheischler, and Bernhard Schölkopf. 2016. Distinguishing cause from effect using observational data: methods and benchmarks. *Journal of Machine Learning Research* 17, 32 (2016), 1–102.
- [29] Ignavier Ng, AmirEmad Ghassami, and Kun Zhang. 2020. On the role of sparsity and dag constraints for learning linear dags. *Advances in Neural Information Processing Systems* 33 (2020), 17943–17954.
- [30] Ignavier Ng, Sébastien Lachapelle, Nan Rosemary Ke, Simon Lacoste-Julien, and Kun Zhang. 2022. On the convergence of continuous constrained optimization for structure learning. In *International Conference on Artificial Intelligence and Statistics*. Pmlr, 8176–8198.
- [31] David A Nix and Andreas S Weigend. 1994. Estimating the mean and variance of the target probability distribution. In *Proceedings of 1994 IEEE international conference on neural networks (ICNN'94)*, Vol. 1. IEEE, 55–60.
- [32] Gunwoong Park. 2020. Identifiability of additive noise models using conditional variances. *Journal of Machine Learning Research* 21, 75 (2020), 1–34.
- [33] Jonas Peters, Joris M Mooij, Dominik Janzing, and Bernhard Schölkopf. 2014. Causal discovery with continuous additive noise models. *The Journal of Machine Learning Research* 15, 1 (2014), 2009–2053.
- [34] Maximilian Seitzer, Arash Tavakoli, Dimitrije Antic, and Georg Martius. 2022. On the Pitfalls of Heteroscedastic Uncertainty Estimation with Probabilistic Neural Networks. In *International Conference on Learning Representations*.
- [35] Eleni Sgouritsa, Dominik Janzing, Philipp Hennig, and Bernhard Schölkopf. 2015. Inference of cause and effect with unsupervised inverse regression. In *Artificial intelligence and statistics*. PMLR, 847–855.
- [36] Shohei Shimizu, Patrik O Hoyer, Aapo Hyvärinen, Antti Kerminen, and Michael Jordan. 2006. A linear non-Gaussian acyclic model for causal discovery. *Journal of Machine Learning Research* 7, 10 (2006).
- [37] Nicki Skafte, Martin Jørgensen, and Søren Hauberg. 2019. Reliable training and estimation of variance networks. *Advances in Neural Information Processing Systems* 32 (2019).
- [38] Peter Spirtes, Clark N Glymour, and Richard Scheines. 2000. *Causation, prediction, and search*. MIT press.
- [39] Peter Spirtes, Christopher Meek, and Thomas Richardson. 1995. Causal inference in the presence of latent variables and selection bias. In *Proceedings of the Eleventh conference on Uncertainty in artificial intelligence*. 499–506.
- [40] Peter Spirtes and Kun Zhang. 2016. Causal discovery and inference: concepts and recent methodological advances. In *Applied informatics*, Vol. 3. Springer, 3.
- [41] Andrew Stirn, Harm Wessels, Megan Schertzer, Laura Pereira, Neville Sanjana, and David Knowles. 2023. Faithful heteroscedastic regression with neural networks. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 5593–5613.
- [42] Marc Teyssier and Daphne Koller. 2005. Ordering-based search: a simple and effective algorithm for learning Bayesian networks. In *Proceedings of the Twenty-First Conference on Uncertainty in Artificial Intelligence*. 584–590.
- [43] Uddeshya Upadhyay, Jae Myung Kim, Cordelia Schmidt, Bernhard Schölkopf, and Zeynep Akata. 2023. Likelihood annealing: Fast calibrated uncertainty for regression. *arXiv preprint arXiv:2302.11012* (2023).
- [44] Yu Wang, An Zhang, Xiang Wang, Yancheng Yuan, Xiangnan He, and Tat-Seng Chua. 2022. Differentiable invariant causal discovery. *arXiv preprint arXiv:2205.15638* (2022).
- [45] Stephen J Wright. 2006. Numerical optimization.
- [46] CF Jeff Wu. 1983. On the convergence properties of the EM algorithm. *The Annals of statistics* (1983), 95–103.
- [47] Sascha Xu, Alexander Marx, Osman Mian, and Jilles Vreeken. 2022. Causal inference with heteroscedastic noise models. In *Proceedings of the AAAI Workshop on Information Theoretic Causal Inference and Discovery*.
- [48] Naiyu Yin, Tian Gao, Yue Yu, and Qiang Ji. 2024. Effective causal discovery under identifiable heteroscedastic noise model. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 16486–16494.
- [49] Yue Yu, Jie Chen, Tian Gao, and Mo Yu. 2019. DAG-GNN: DAG structure learning with graph neural networks. In *International conference on machine learning*. PMLR, 7154–7163.
- [50] Yue Yu, Tian Gao, Naiyu Yin, and Qiang Ji. 2021. DAGs with no curl: An efficient DAG structure learning approach. In *International conference on machine learning*. Pmlr, 12156–12166.
- [51] An Zhang, Fangfu Liu, Wenchang Ma, Zhibo Cai, Xiang Wang, and Tat-Seng Chua. 2023. Boosting Causal Discovery via Adaptive Sample Reweighting. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=LNpMtk15AS4>
- [52] An Zhang, Fangfu Liu, Wenchang Ma, Zhibo Cai, Xiang Wang, and Tat-Seng Chua. 2023. Boosting differentiable causal discovery via adaptive sample reweighting. *arXiv preprint arXiv:2303.03187* (2023).

- [53] Kun Zhang and Aapo Hyvarinen. 2012. On the identifiability of the post-nonlinear causal model. *arXiv preprint arXiv:1205.2599* (2012).
- [54] Xun Zheng, Bryon Aragam, Pradeep K Ravikumar, and Eric P Xing. 2018. Dags with no tears: Continuous optimization for structure learning. *Advances in neural information processing systems* 31 (2018).
- [55] Xun Zheng, Chen Dan, Bryon Aragam, Pradeep Ravikumar, and Eric Xing. 2020. Learning sparse nonparametric dags. In *International Conference on Artificial Intelligence and Statistics*. Pmlr, 3414–3425.
- [56] Wanqi Zhou, Shuanghao Bai, Yuqing Xie, Yicong He, Qibin Zhao, and Badong Chen. 2025. An information-theoretic approach for heterogeneous differentiable causal discovery. *Neural Networks* 188 (2025), 107417.